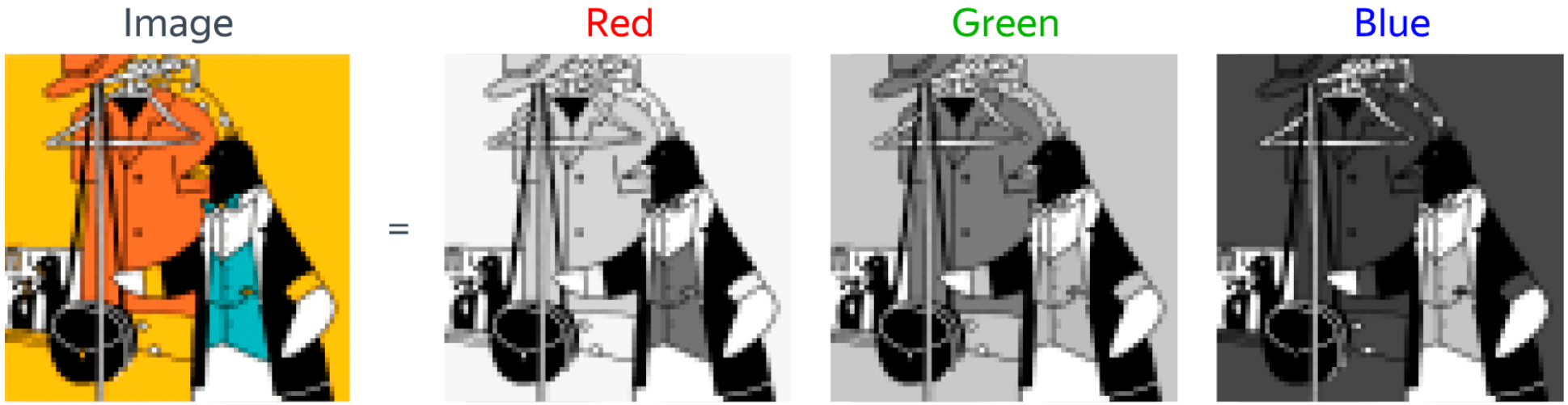
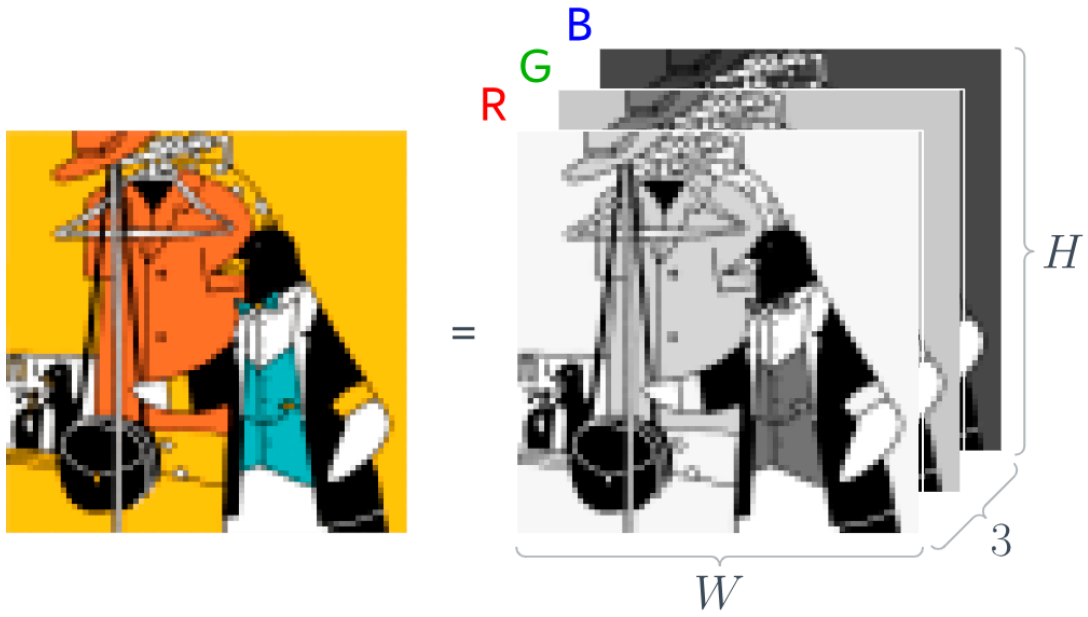
Neural Networks: Computer Vision

Inspired by S. Prince

Image Representation





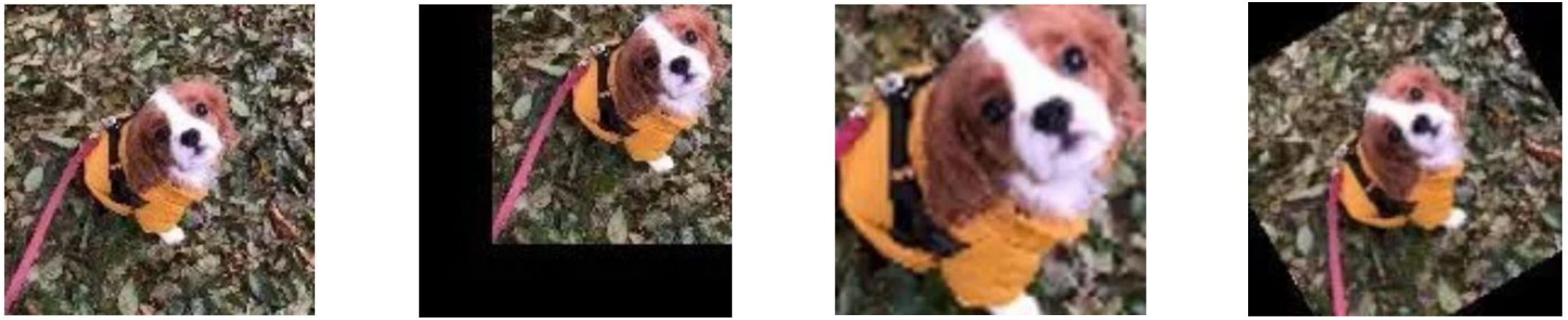
Why MLP is Bad for Images?

How can we work with an images (tensors)?

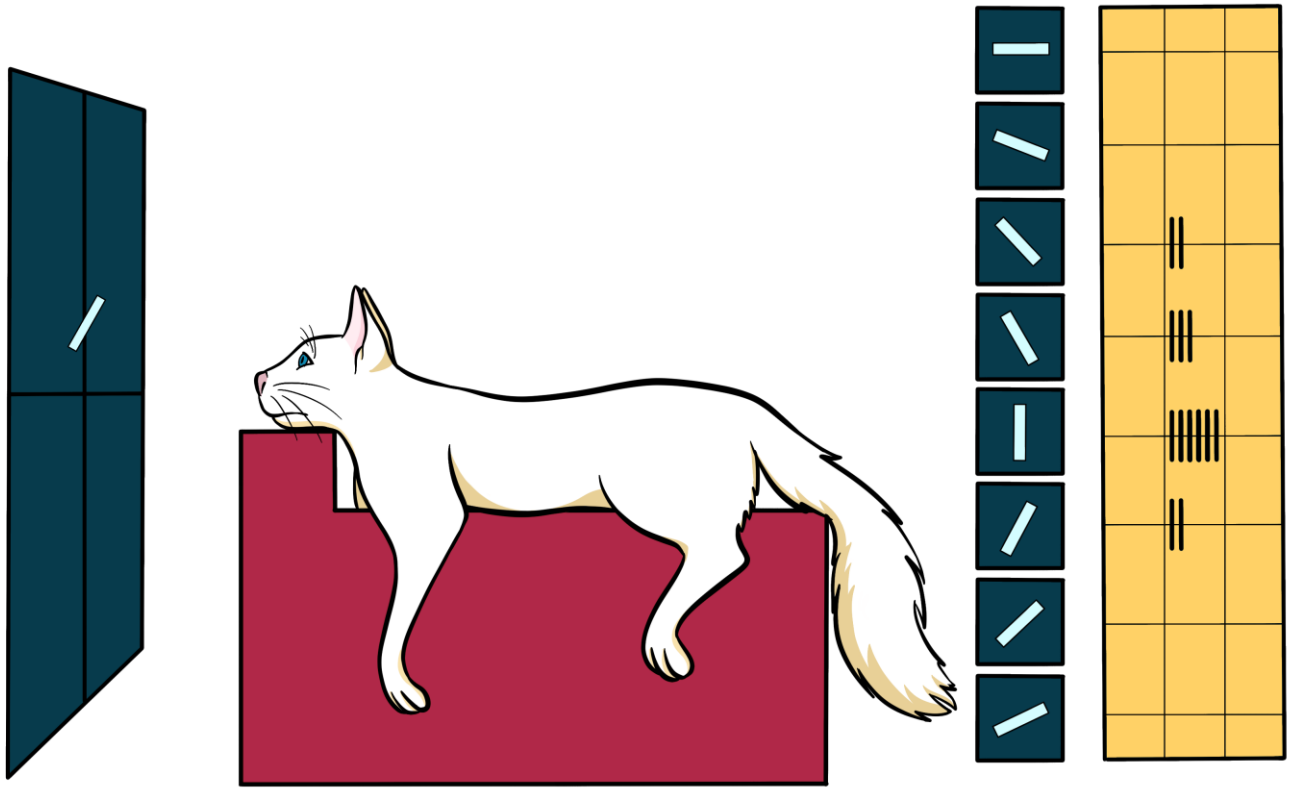
Problems:

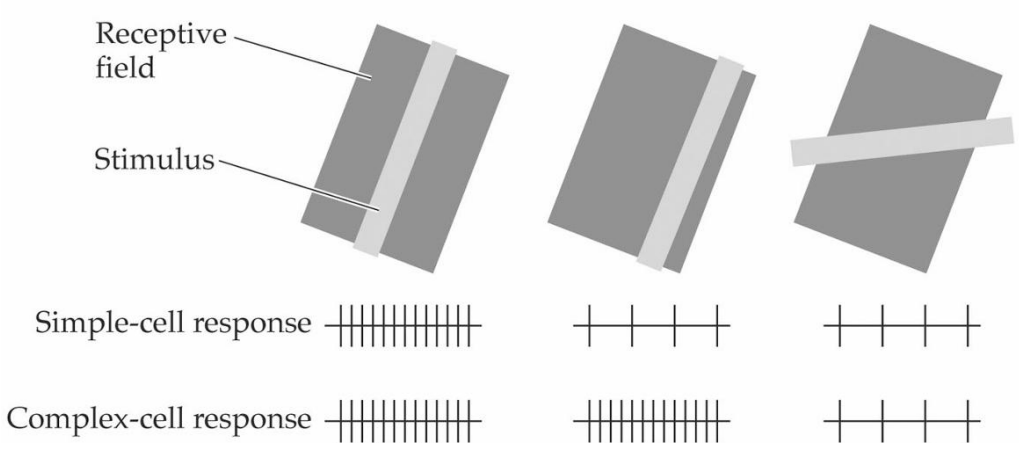
- Number of parameters
- Respect to the data structure (leads to the point above)

What is the data structure?



Biological Idea





David H. Hubel and Torsten N. Wiesel

Kernels

−1	−1	−1
2	2	2
−1	−1	−1

Horizontal

2	−1	−1
−1	2	−1
−1	−1	2

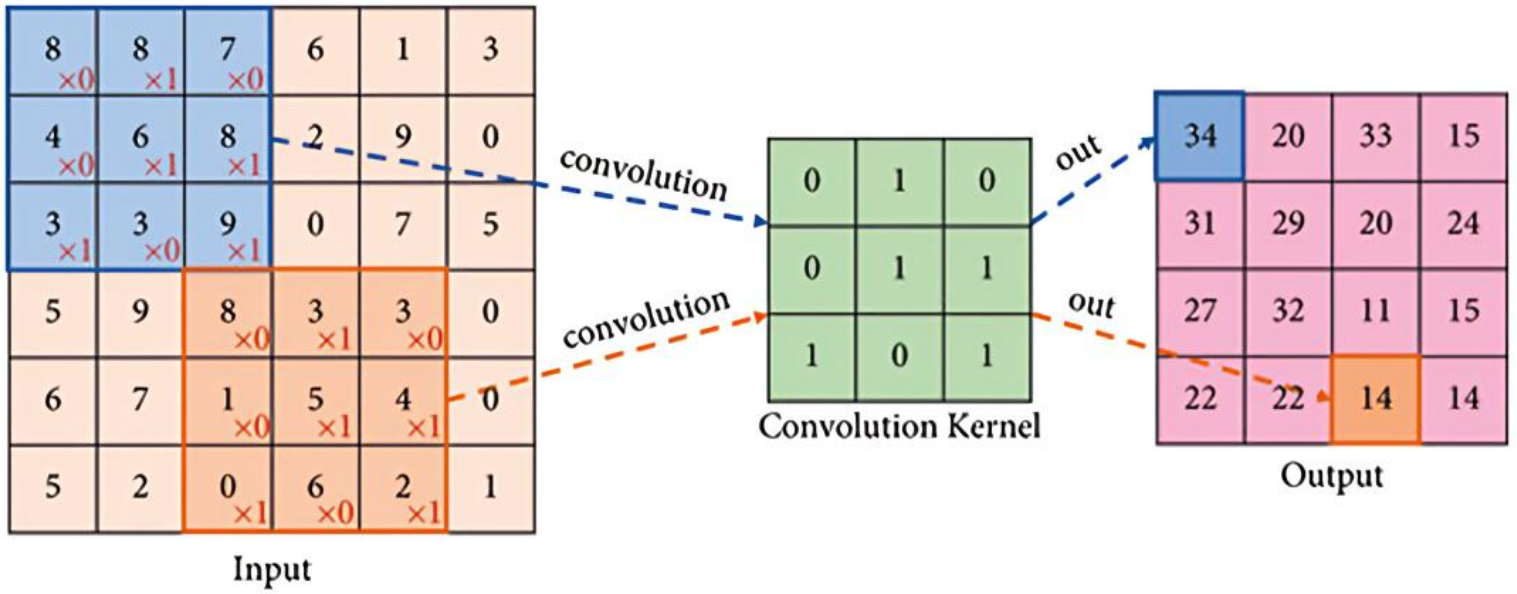
+45°

−1	2	−1
−1	2	−1
−1	2	−1

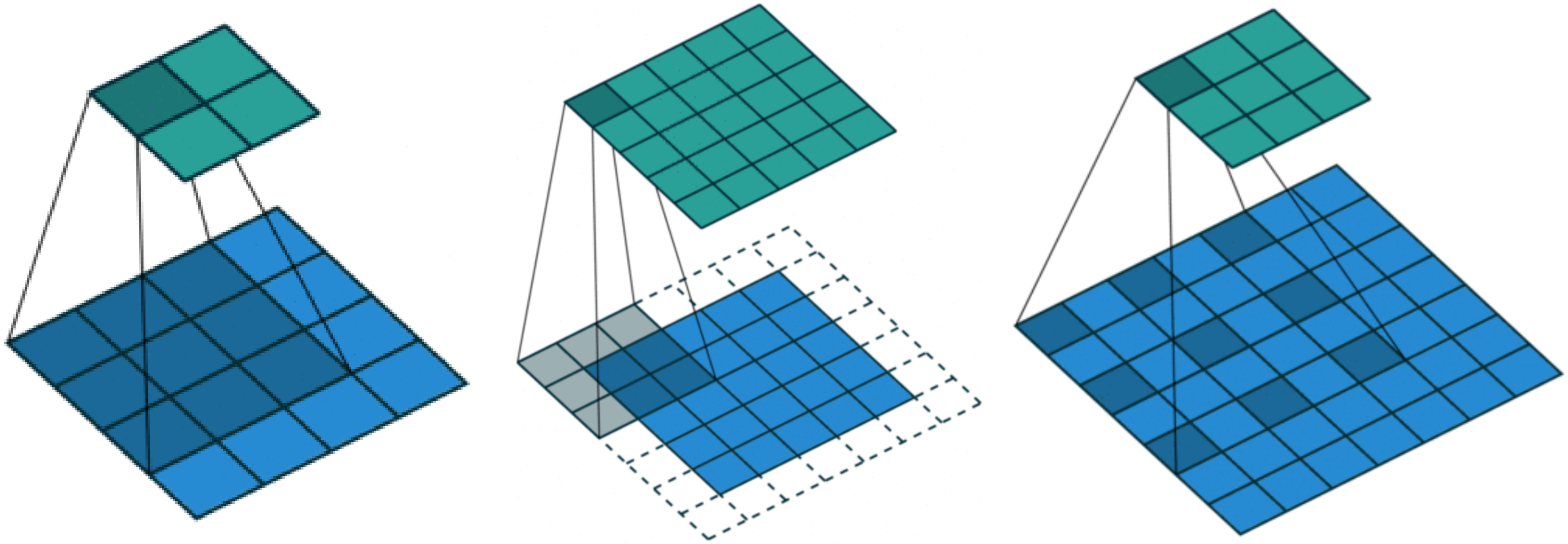
Vertical

−1	−1	2
−1	2	−1
2	−1	−1

−45°



Kernels



Kernels

−1	−1	−1
2	2	2
−1	−1	−1

Horizontal

2	−1	−1
−1	2	−1
−1	−1	2

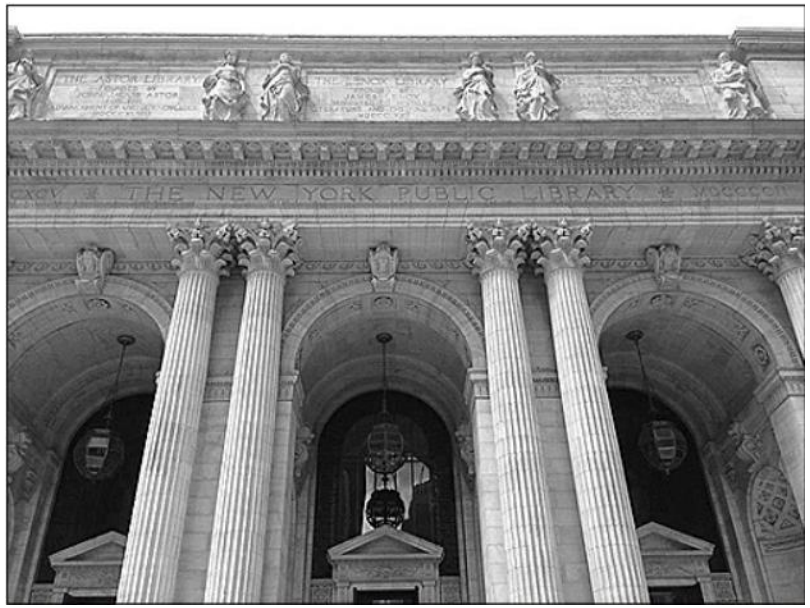
+45°

−1	2	−1
−1	2	−1
−1	2	−1

Vertical

−1	−1	2
−1	2	−1
2	−1	−1

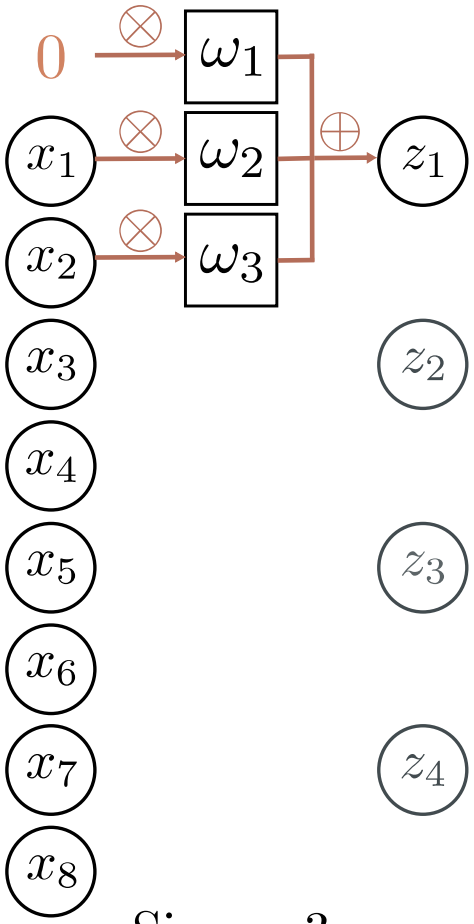
−45°



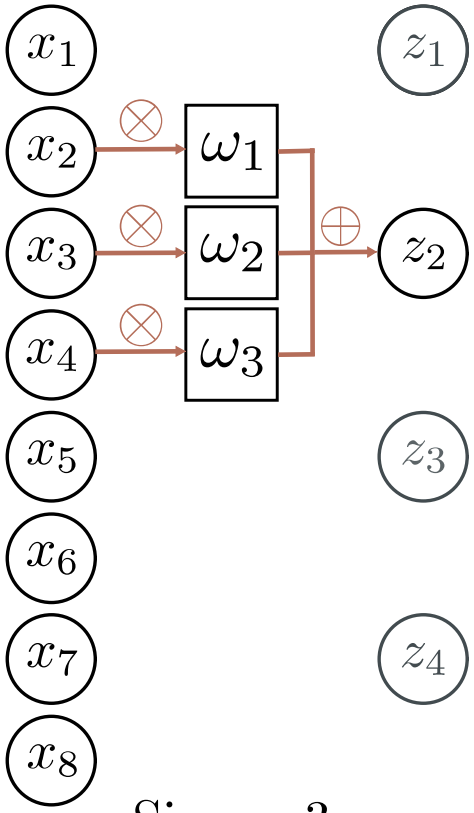
Stride, kernel size, and dilation

- **Stride** = shift by k positions for each output
  - Decreases size of output relative to input
- **Kernel size** = weight a different number of inputs for each output
  - Combine information from a larger area
- **Dilated** convolutions = intersperse kernel values with zeros
  - Combine information from a larger area
  - Fewer parameters

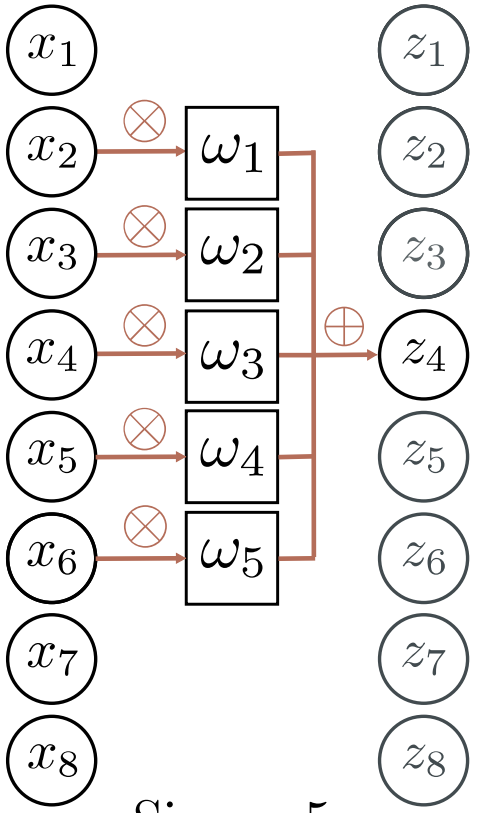
1D Example



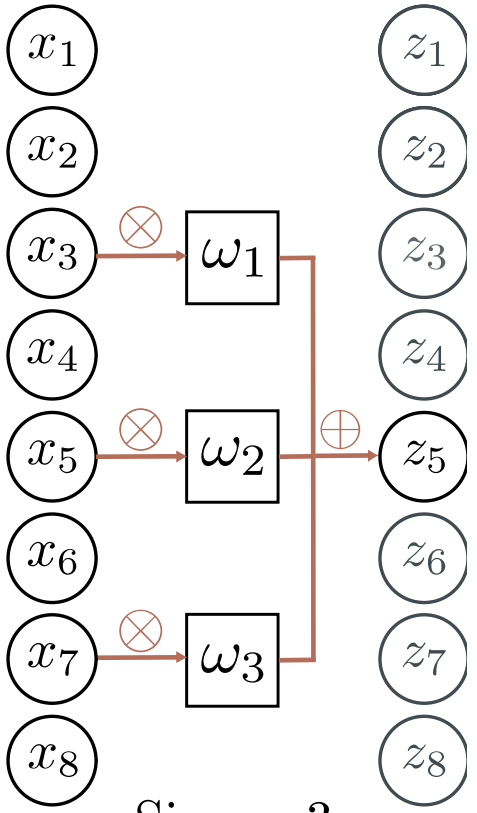
Size = 3
Stride = 2
Dilation = 1



Size = 3
Stride = 2
Dilation = 1



Size = 5
Stride = 1
Dilation = 1



Size = 3
Stride = 1
Dilation = 2

Special Case of Fully-Connected Network

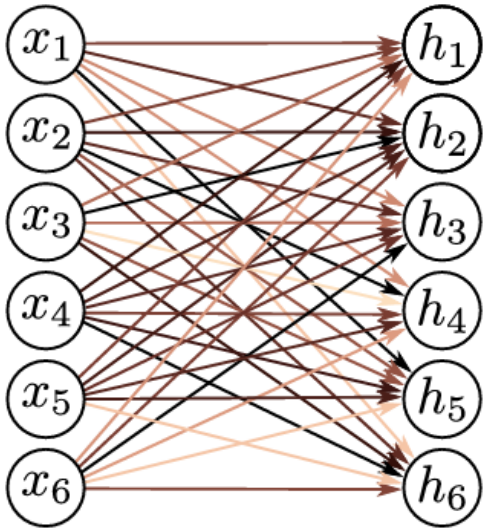
- Convolutional network: 3 weights, 1 bias

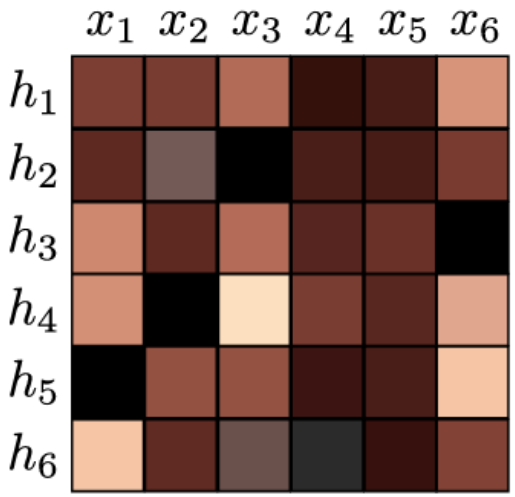
$$h_i = \sigma(w_1 x_{i-1} + w_2 x_i + w_3 x_{i+1} + \beta)$$

- Fully-connected network: D^2 weights, D biases

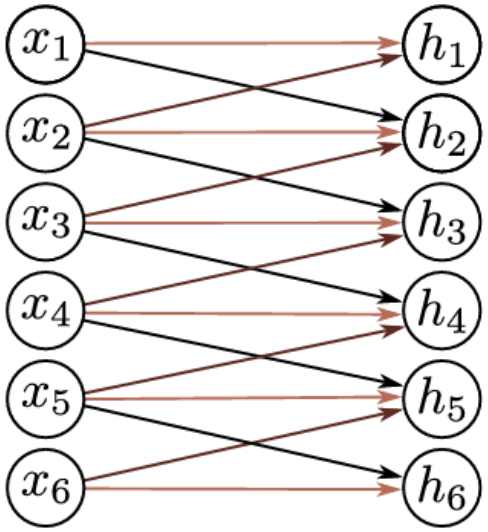
$$h_i = \sigma\left(\sum_{j=0}^{D} w_{i,j} x_j\right)$$

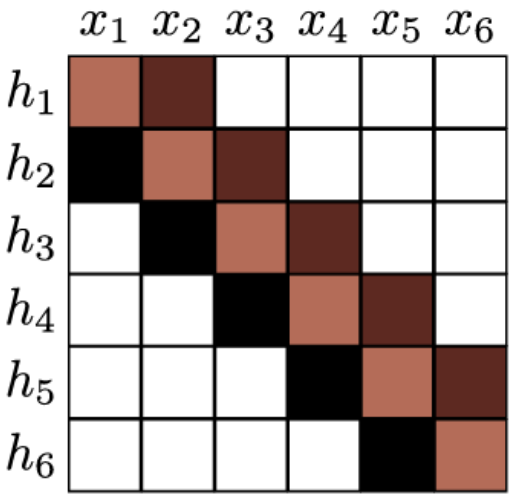
Special Case of Fully-Connected Network



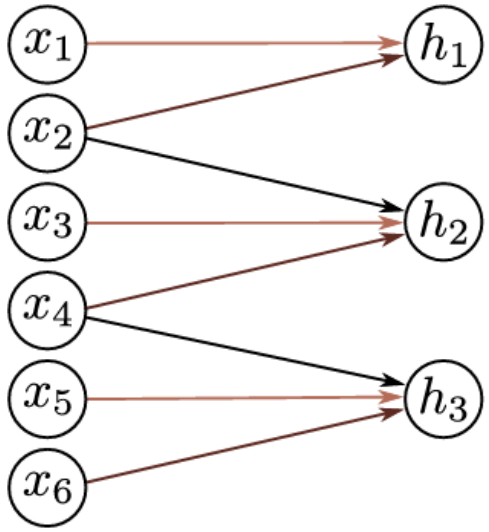


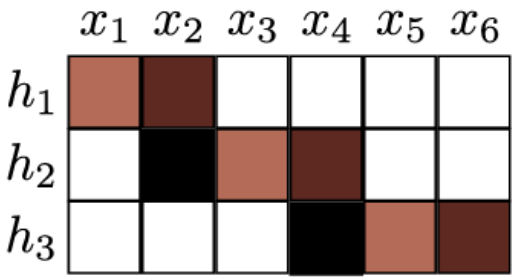
Fully connected network





Convolution, size 3, stride 1, dilation 1, zero padding

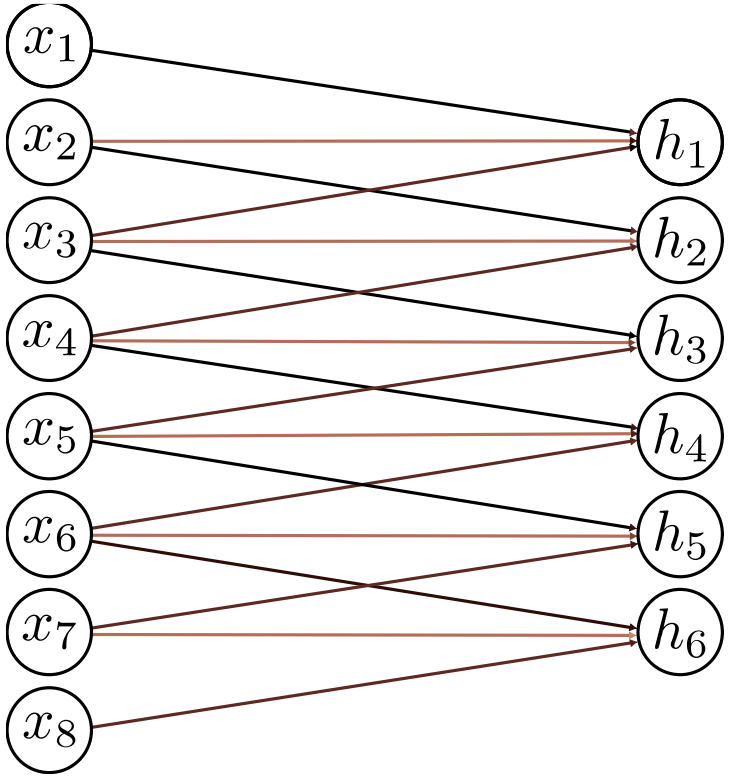


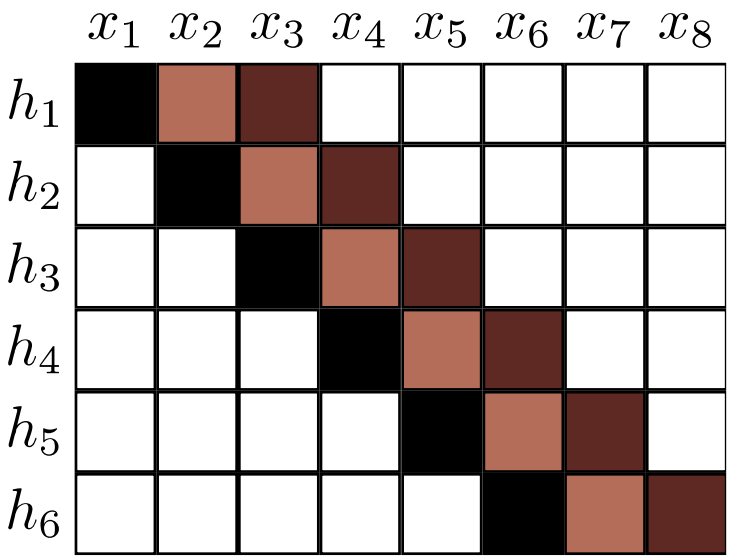


Convolution, size 3, stride 2, dilation 1, zero padding

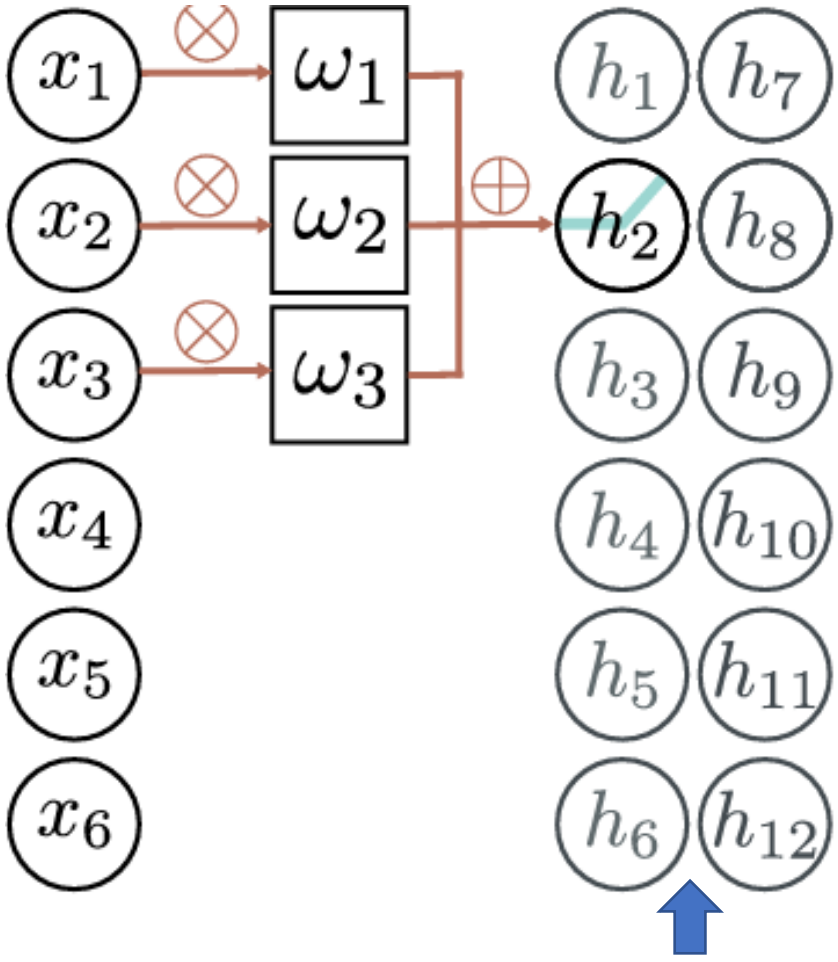
Question

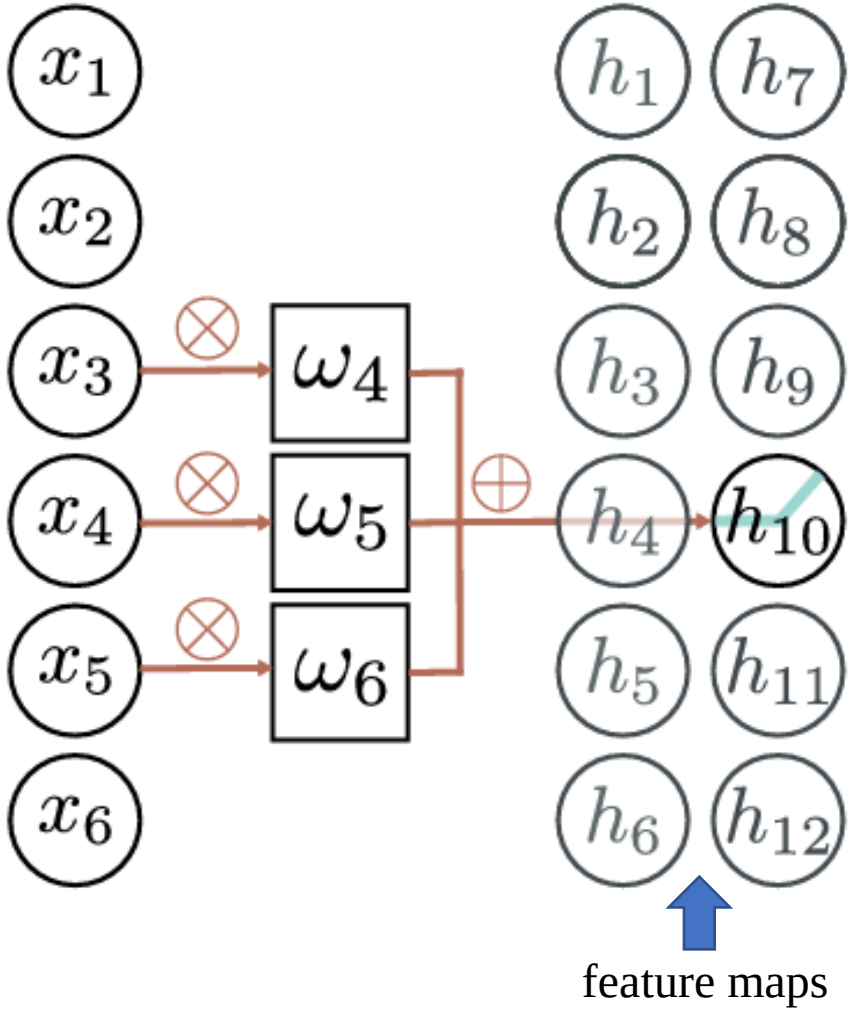
- Kernel size?
- Stride?
- Dilation?
- Zero padding / valid?



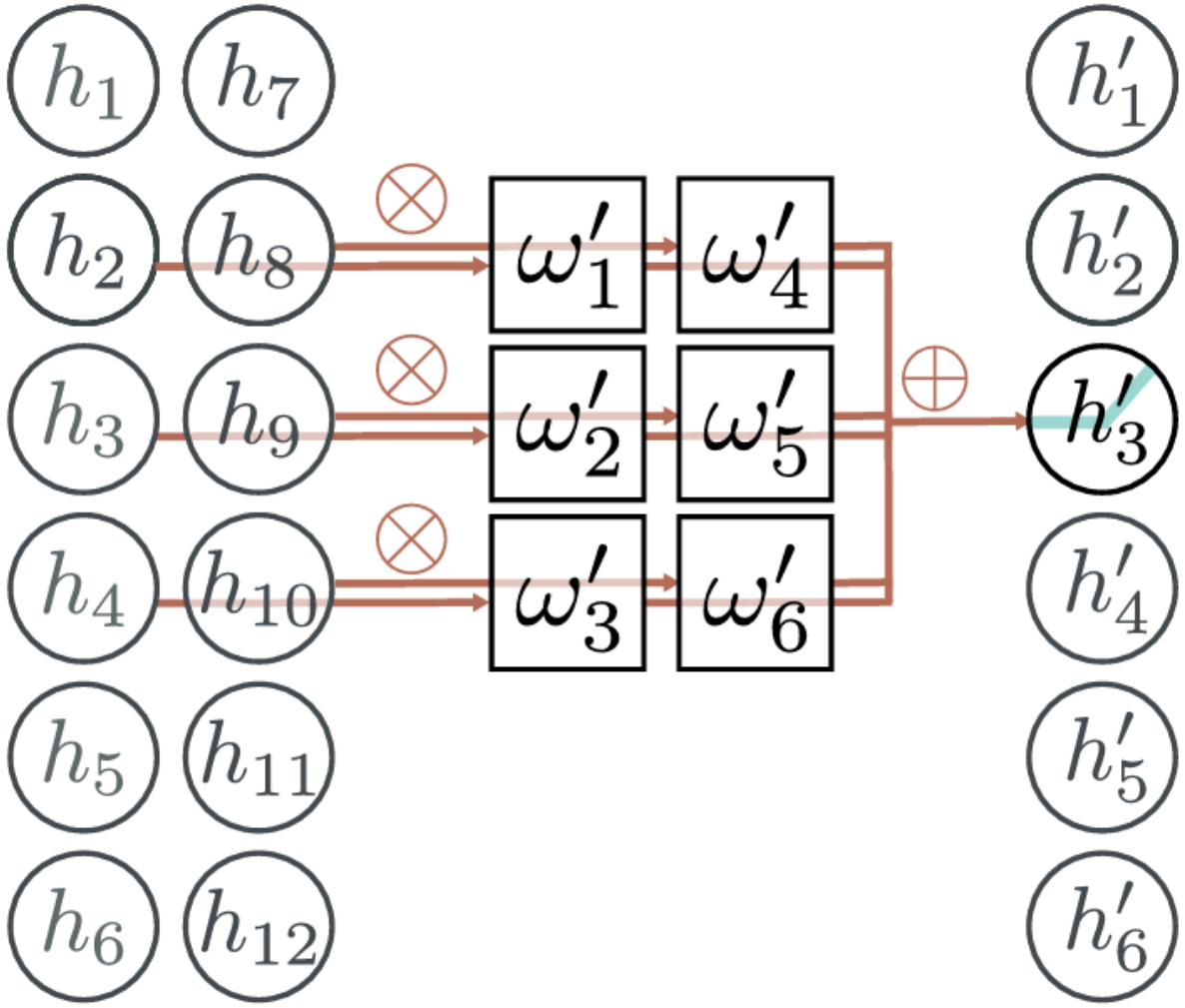


Two output channels, one input channel

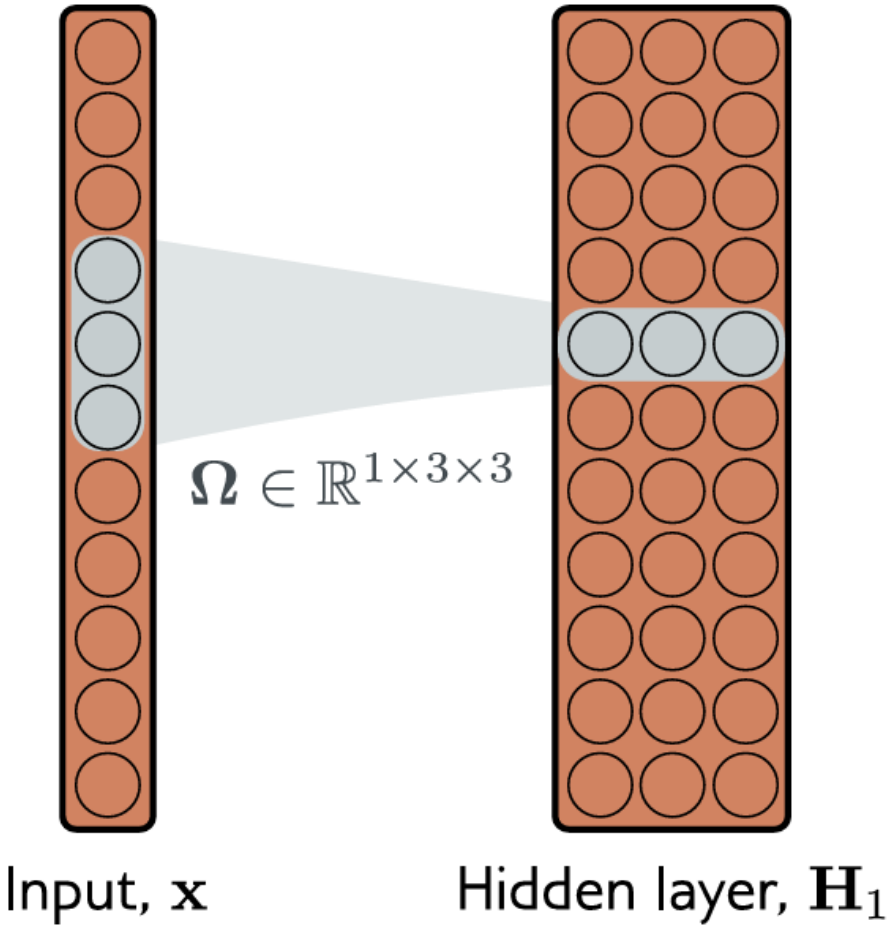




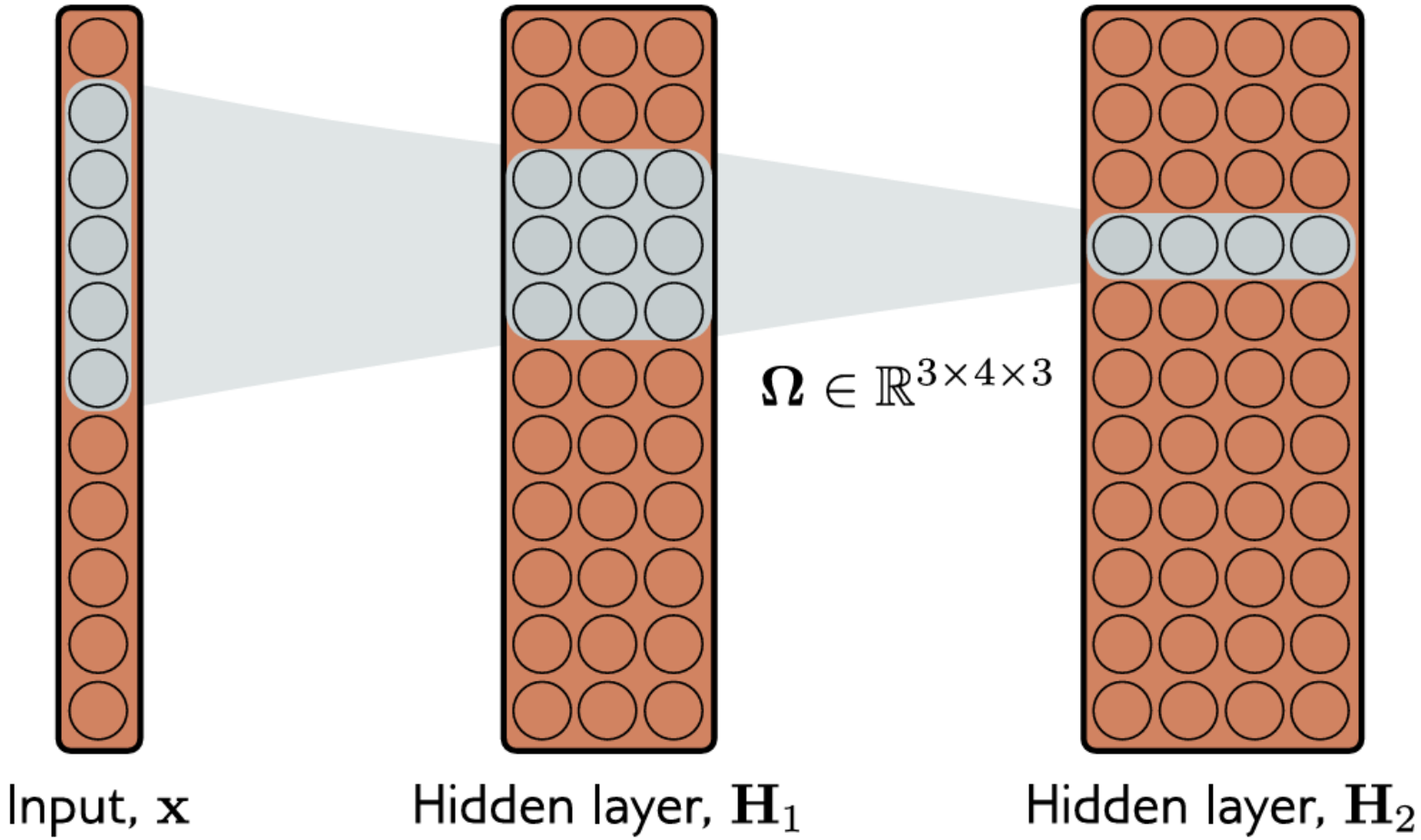
Two input channels, one output channel



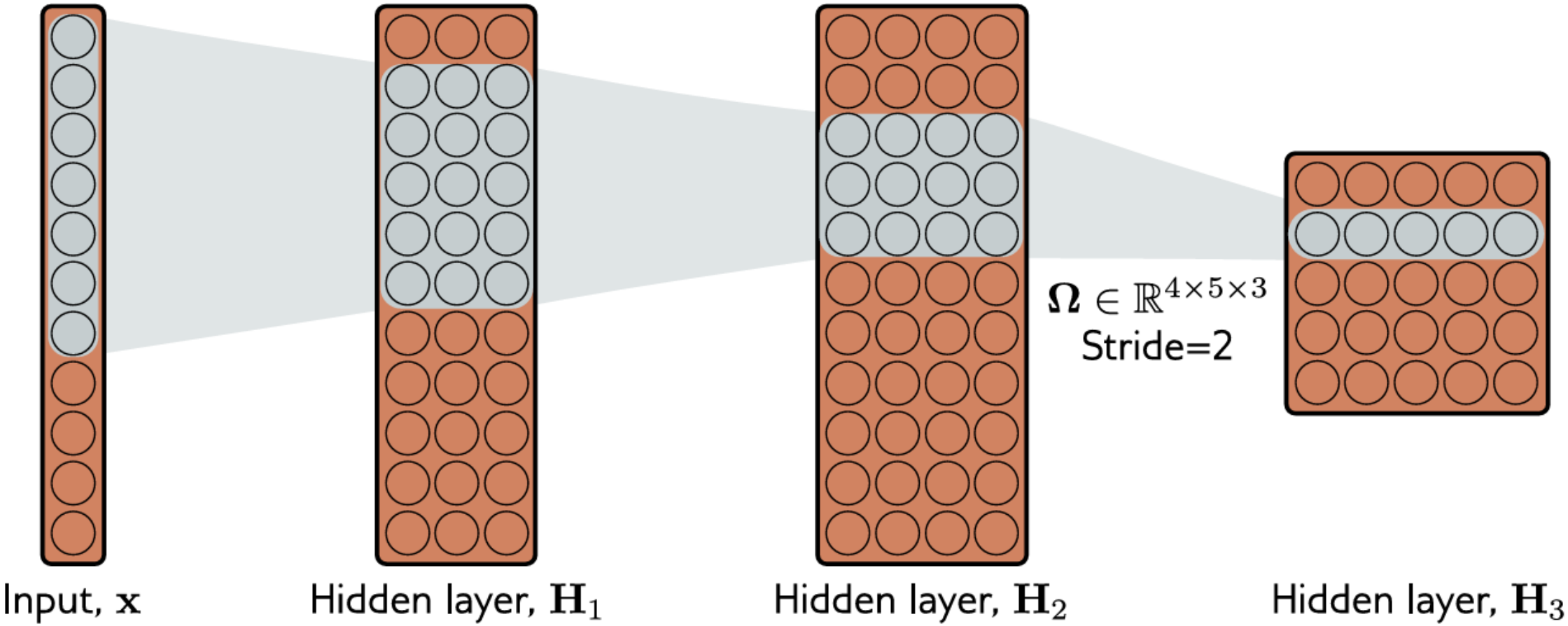
Receptive Fields



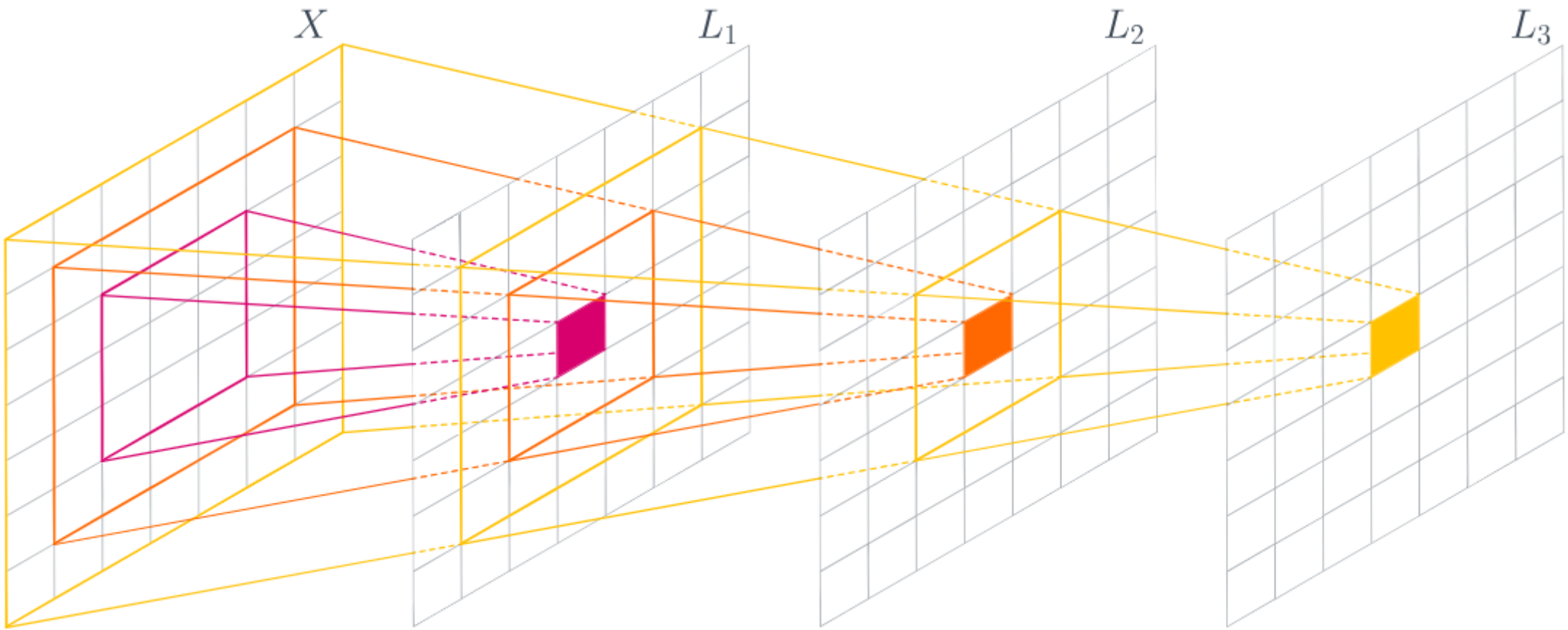
Receptive Fields



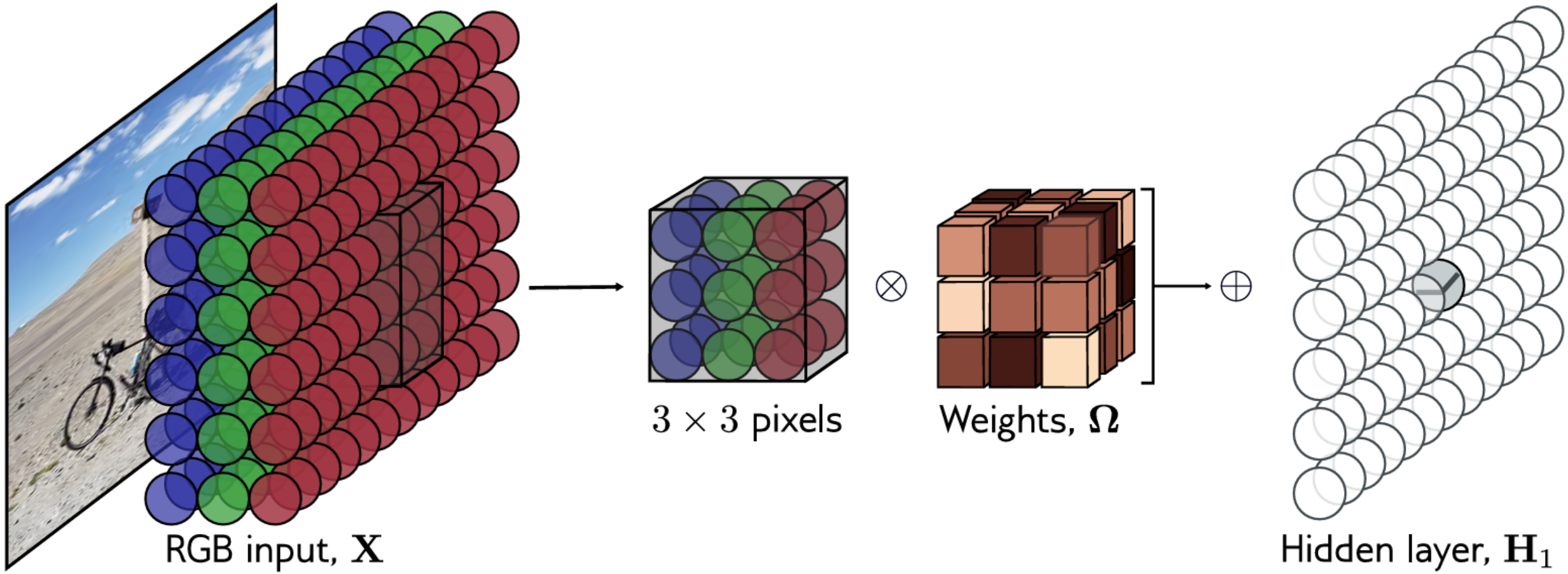
Receptive Fields



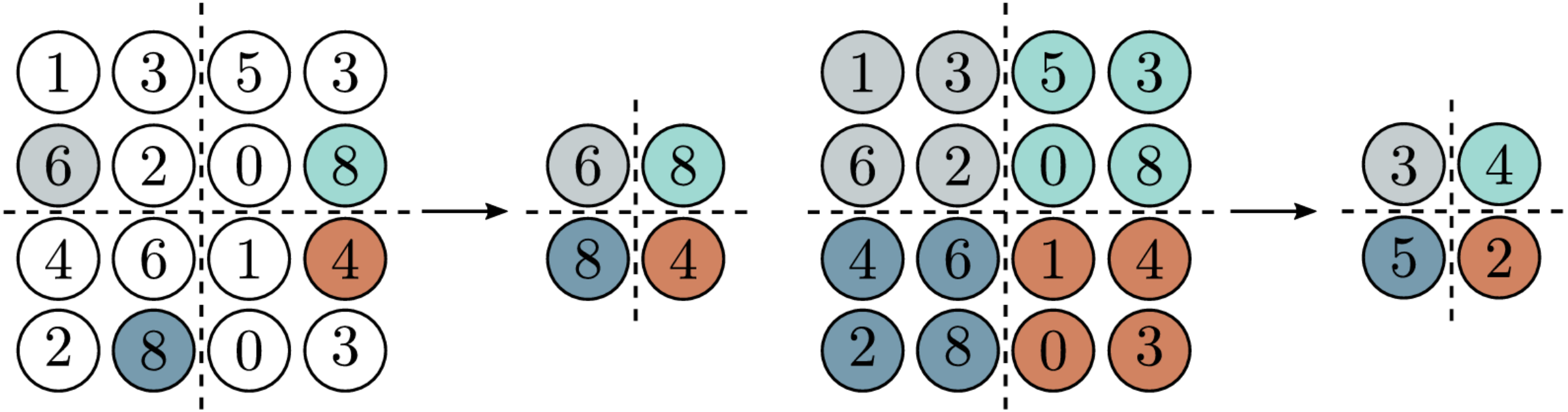
Receptive Fields



Again 2D


RGB input, $\mathbf{X}$
3×3 pixels
Weights, $\mathbf{\Omega}$
Hidden layer, $\mathbf{H}_1$

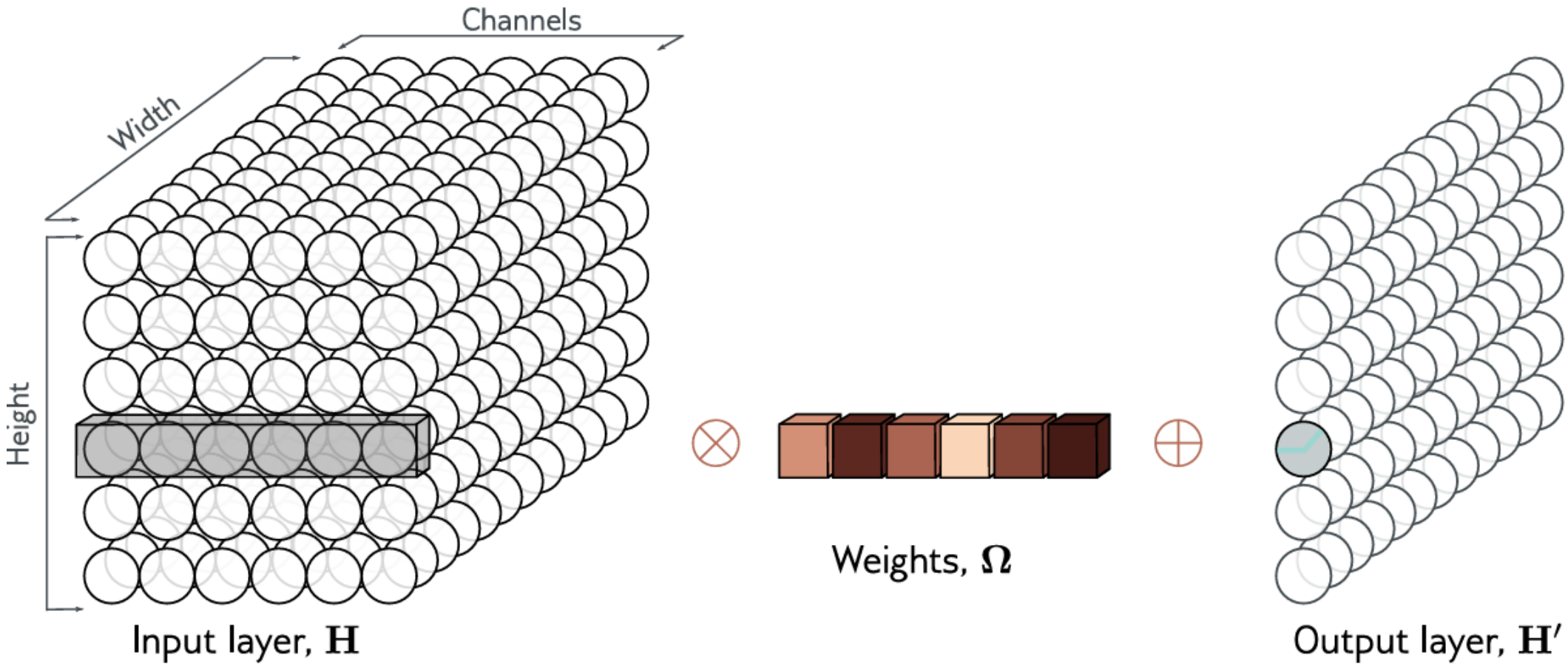
Downsampling



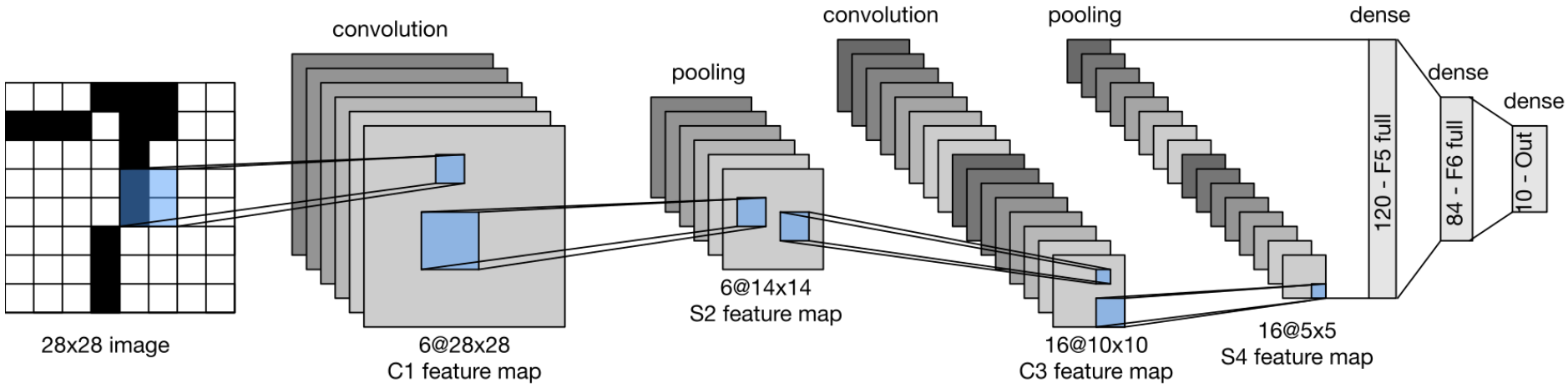
Max pooling
(partial invariance to translation)

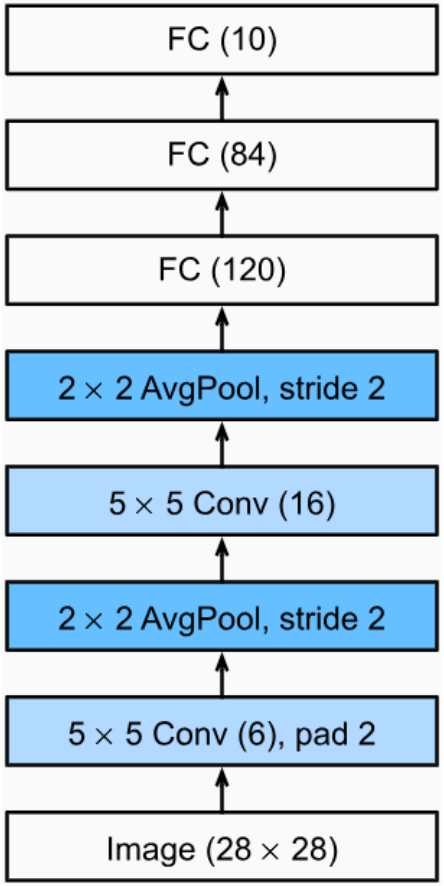
Average pooling

1x1 Convolution



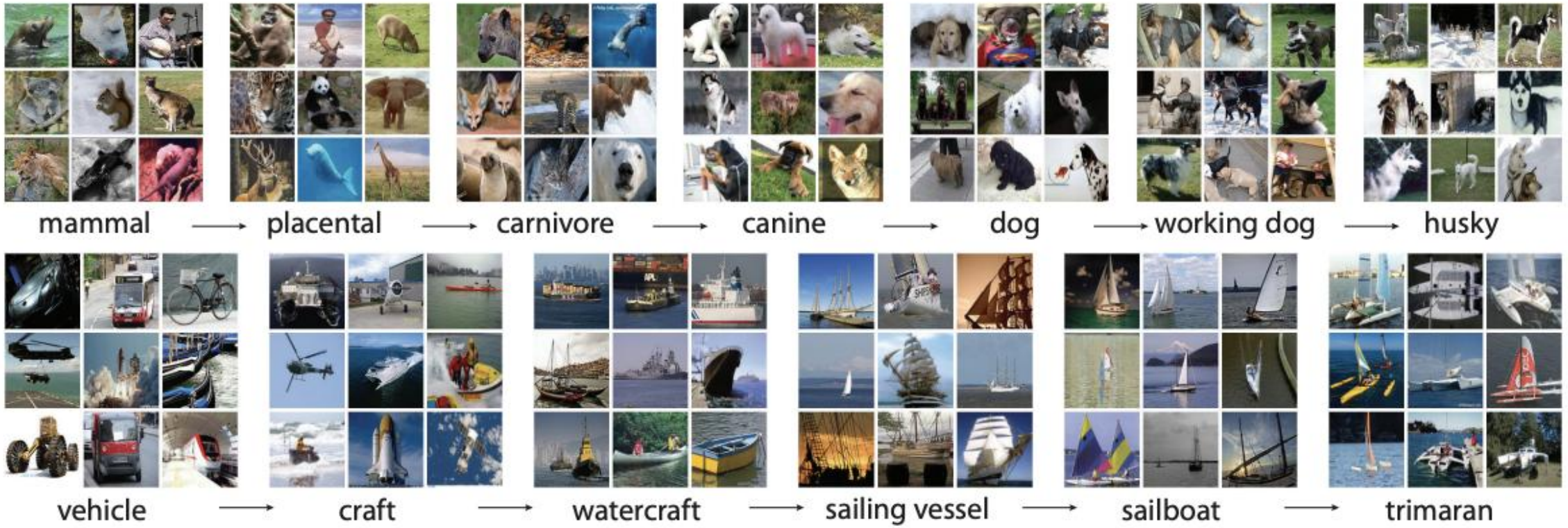
LeNet-5 (1995)





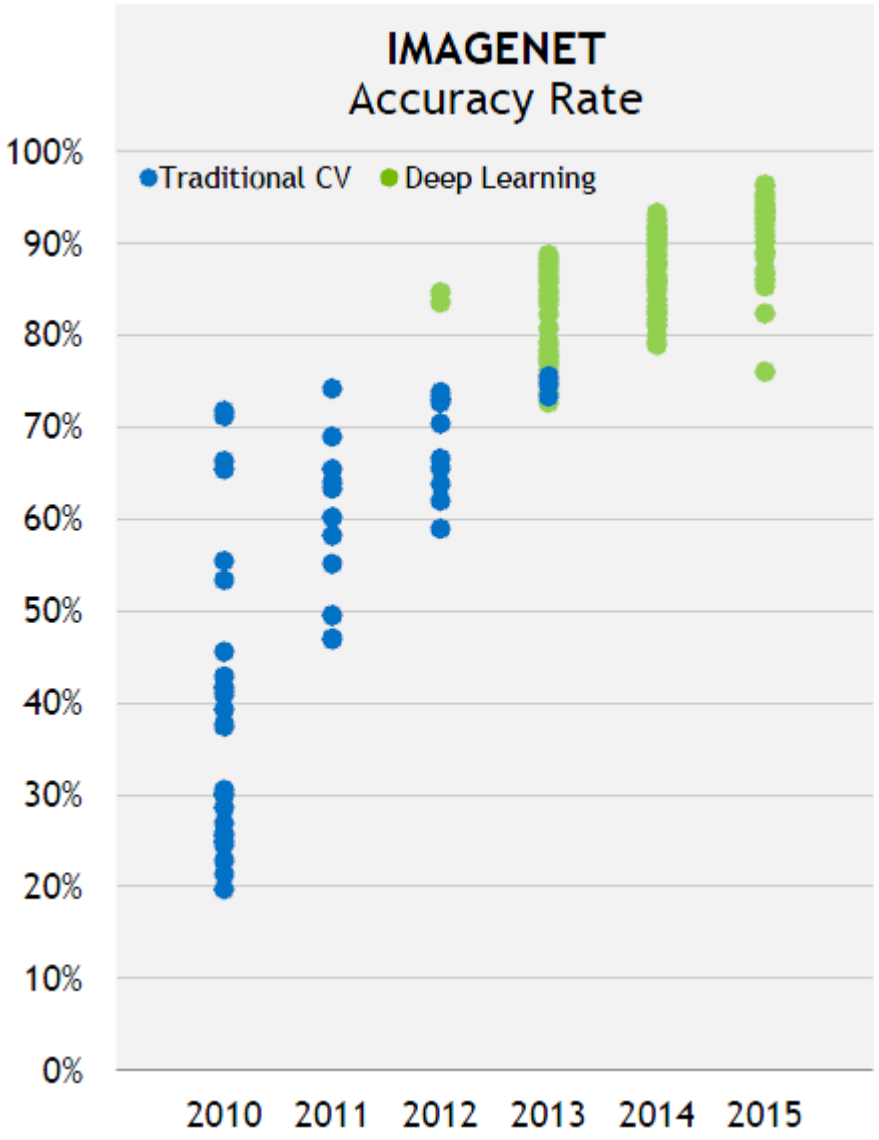
LeNet-5 was trained for about 20 epoches over MNIST. It took 2 to 3 days of CPU time on a Silicon Graphics Origin 2000 server, using a single 200 MHz R10000 processor.
Has ~61 470 parameters.

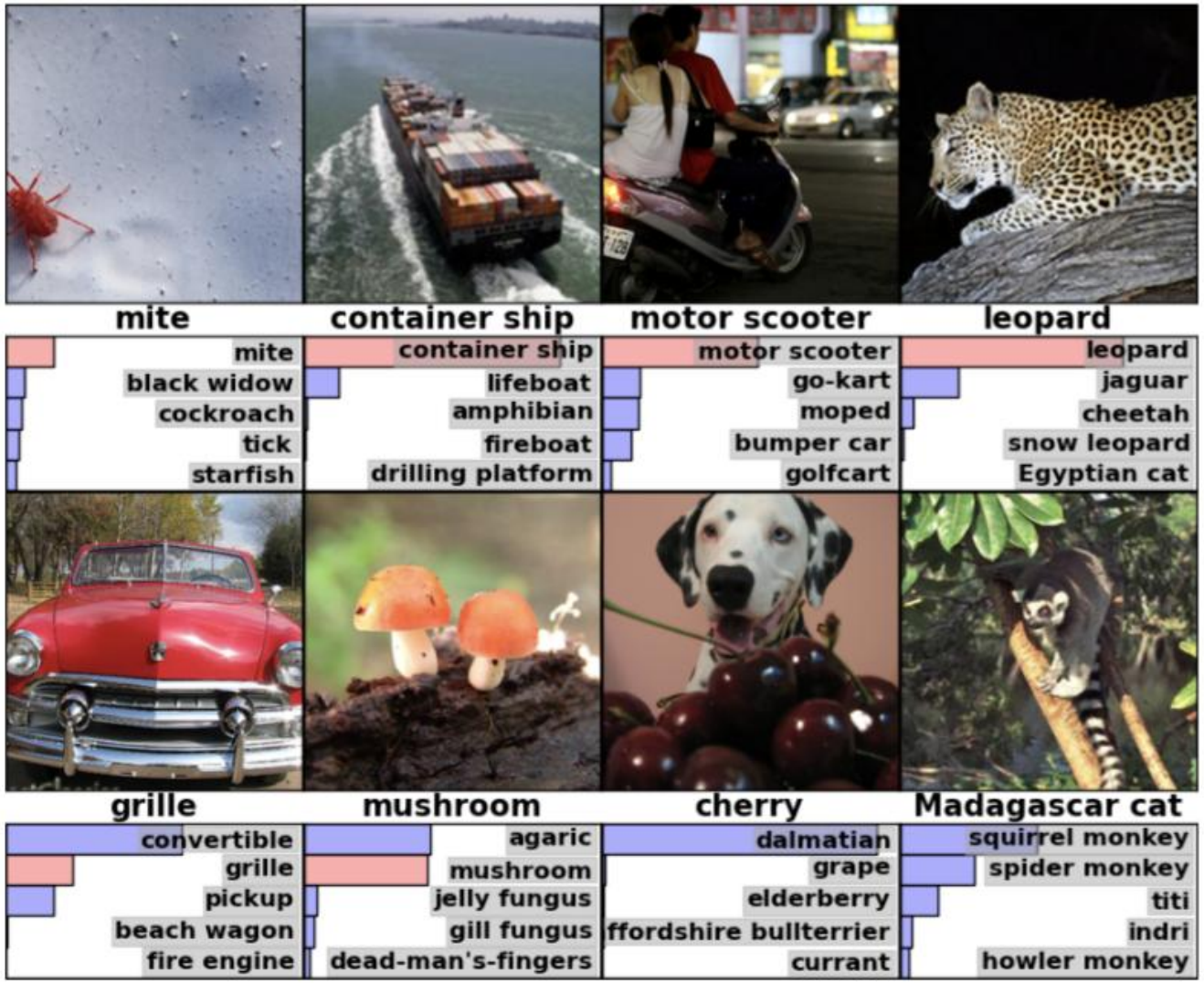
ImageNet Challenge



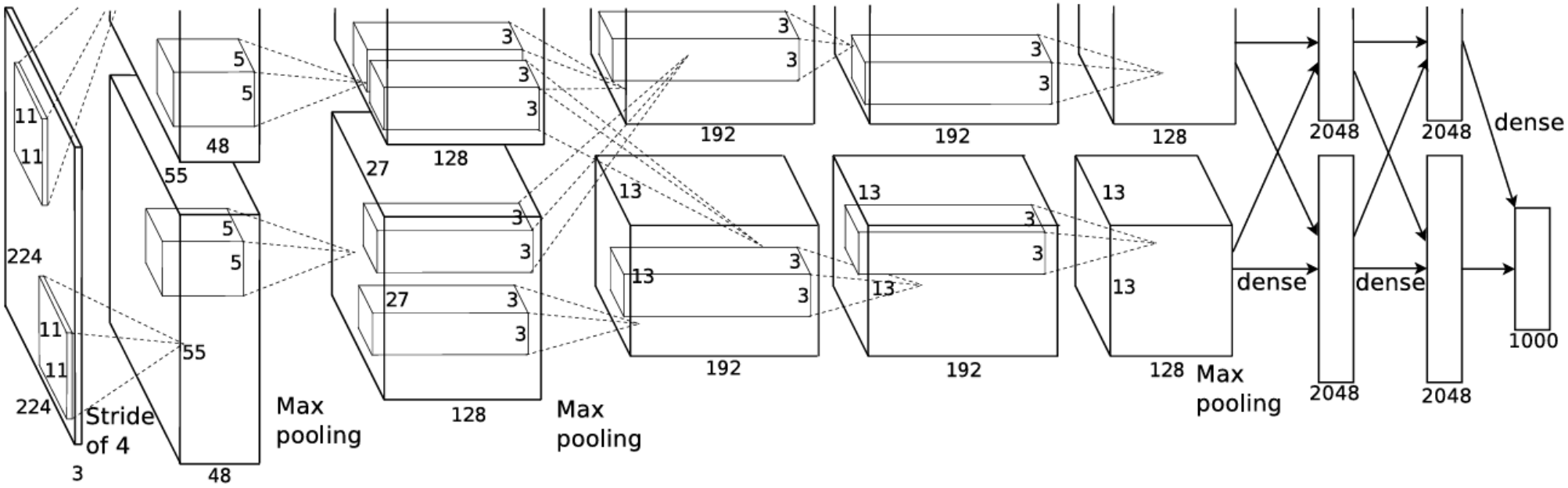
- 1000 object classes (categories)
- Images: 1.2 M train, 100k test

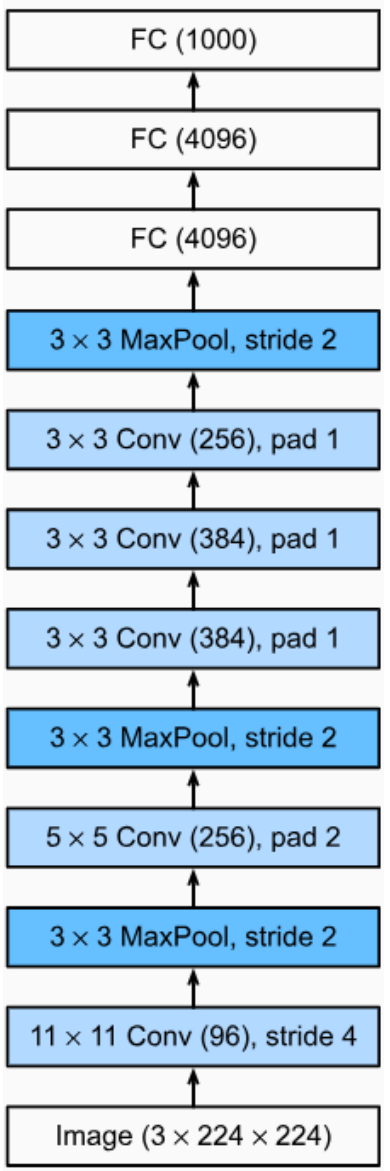
ImageNet Challenge





AlexNet (2012)

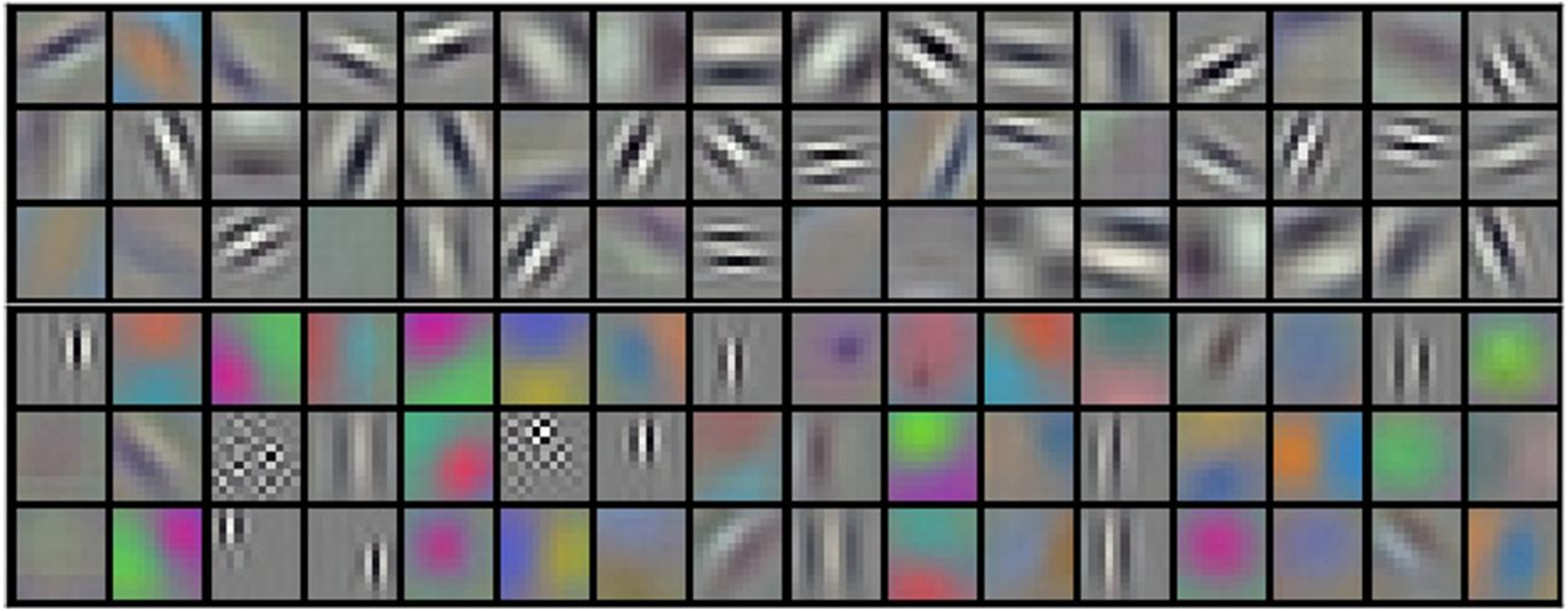




~100 million parameters (~60 millions in dense layers)

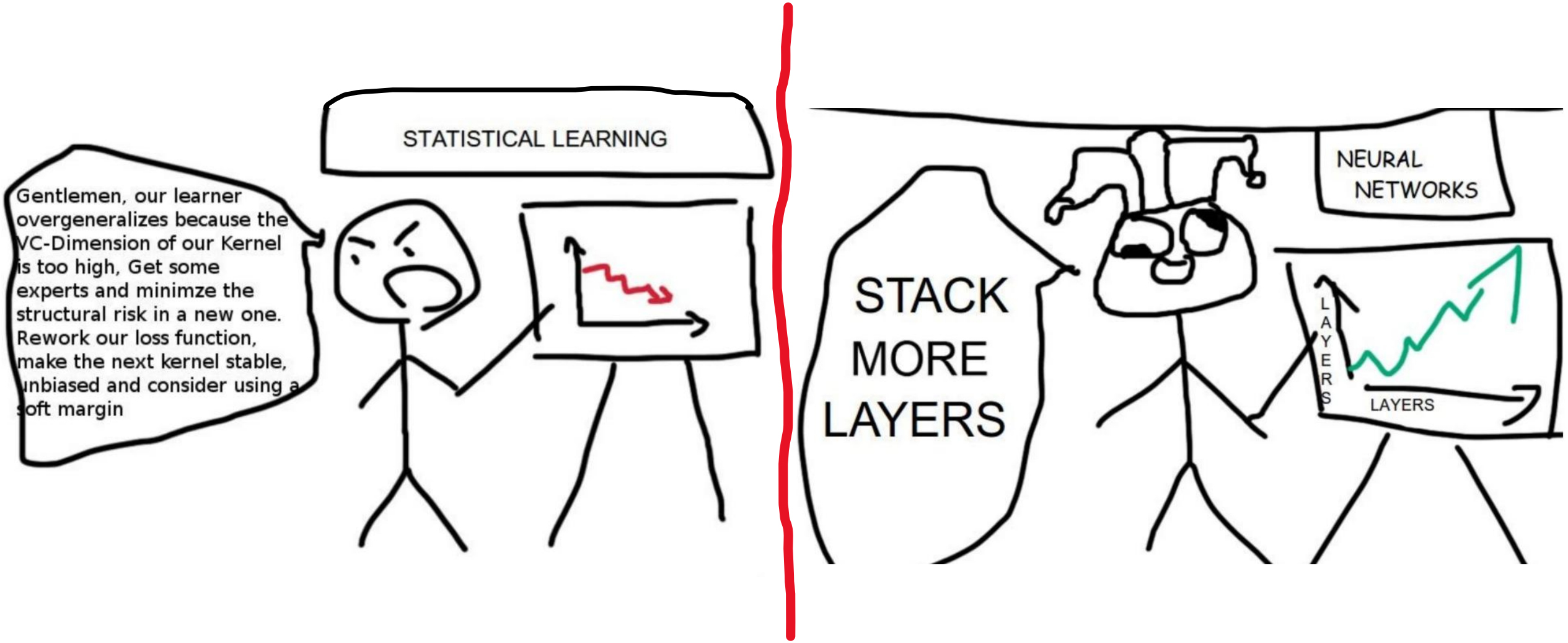
- Augmentations
  Random patches 224x224 (and their horizontal reflections) were taken from 256x256 images
- Subtract average pixel value from each pixel
- ReLU
- Dropout (0.5)
- Batch size 128, SGD with momentum (0.9), L2 weight decay (λ = 0.0005)

AlexNet: First Convolution Kernels

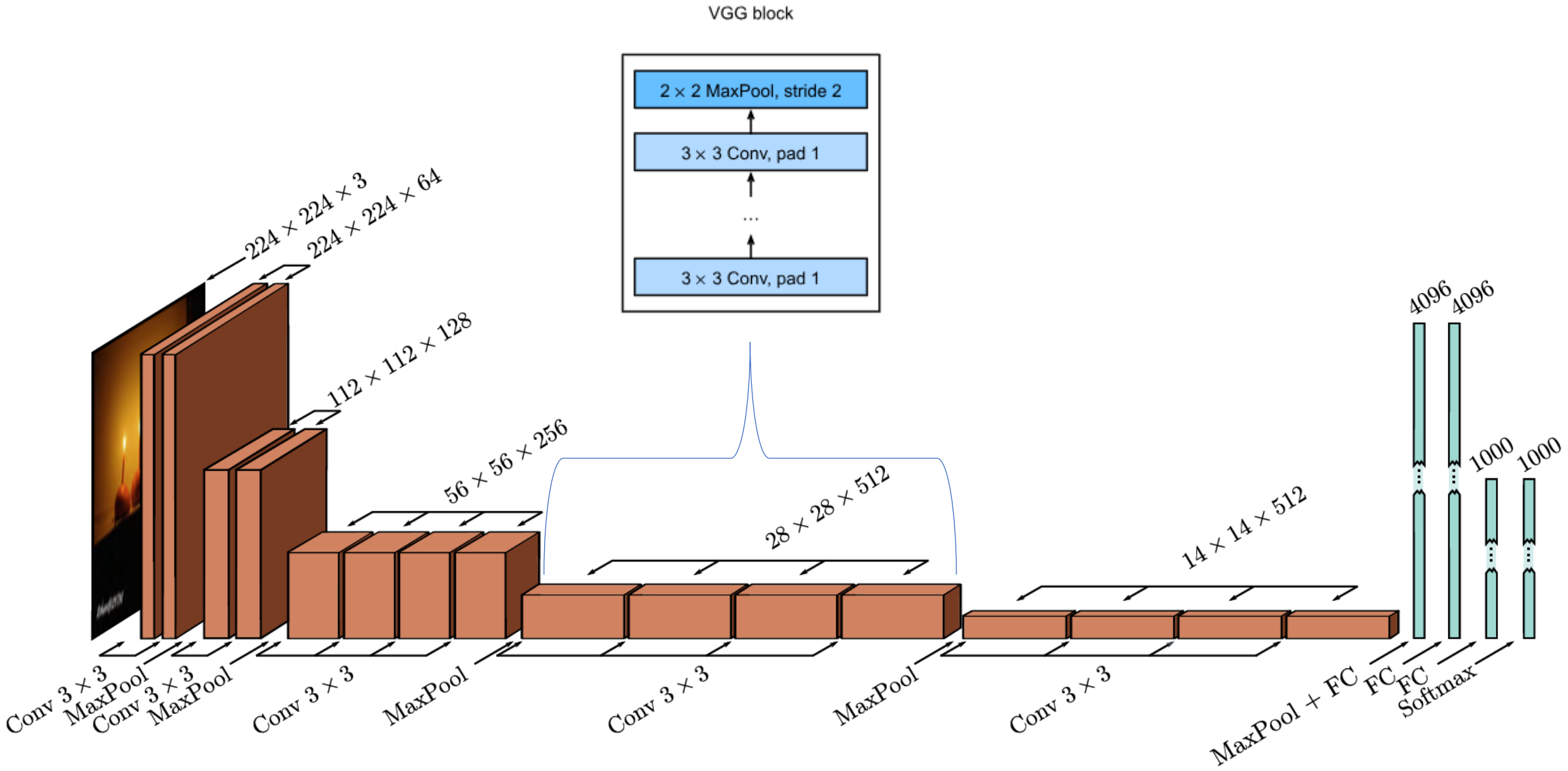


“The kernels on GPU 1 are largely color-agnostic, while the kernels on on GPU 2 are largely color-specific. This kind of specialization occurs during every run and is independent of any particular random weight initialization”

ImageNet Challenge



VGGNet: Very Deep Convolutional Networks (2014)



VGGNet: Configurations

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

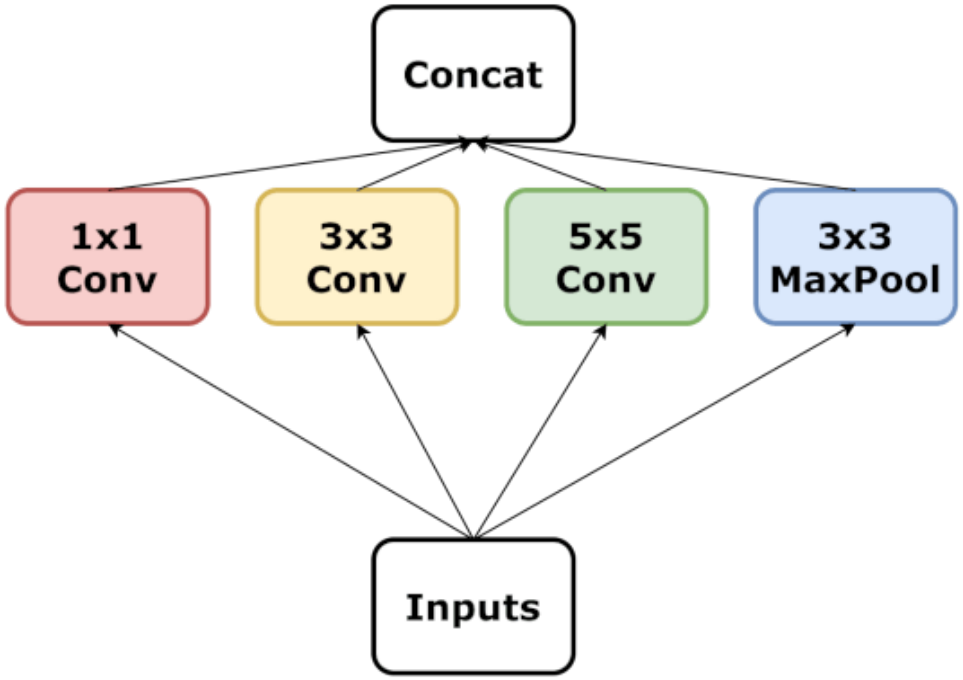
Numbers of parameters (in millions)

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

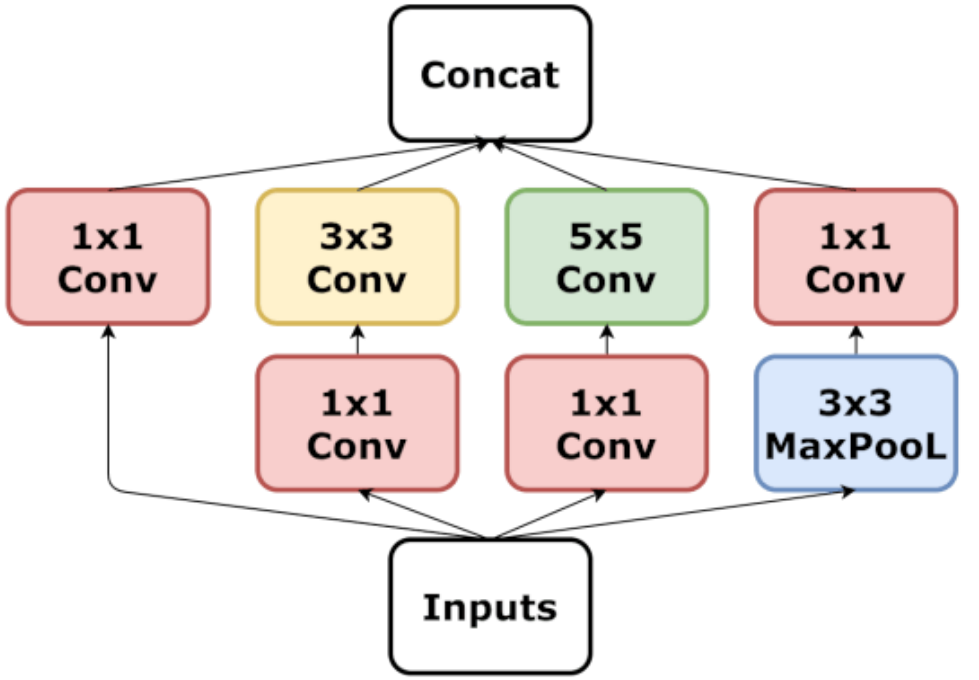
ImageNet Challenge result: 7.3% (2nd)

Inception (Google, 2014)

"We need to go deeper"
from Inception

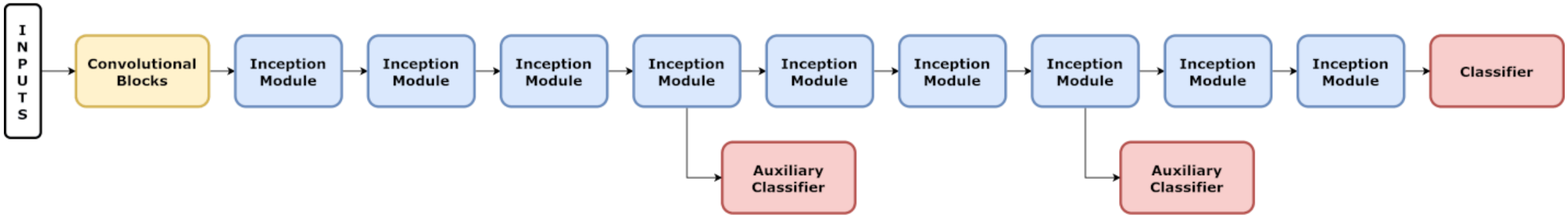


Inception Module



Inception Module with Dimension Reduction

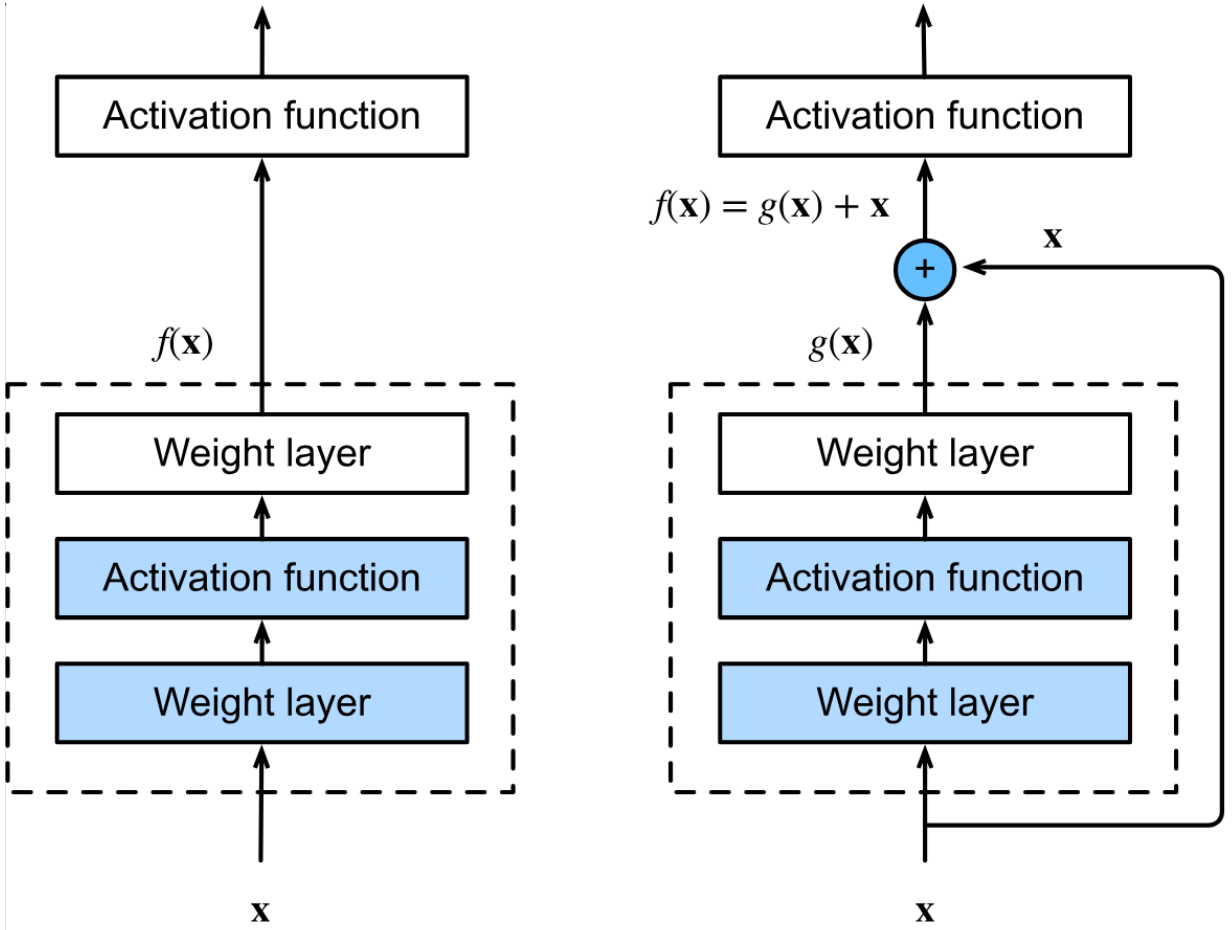
GoogLeNet



$$Loss = 0.3L_{aux,1} + 0.3L_{aux,2} + L_{real}$$

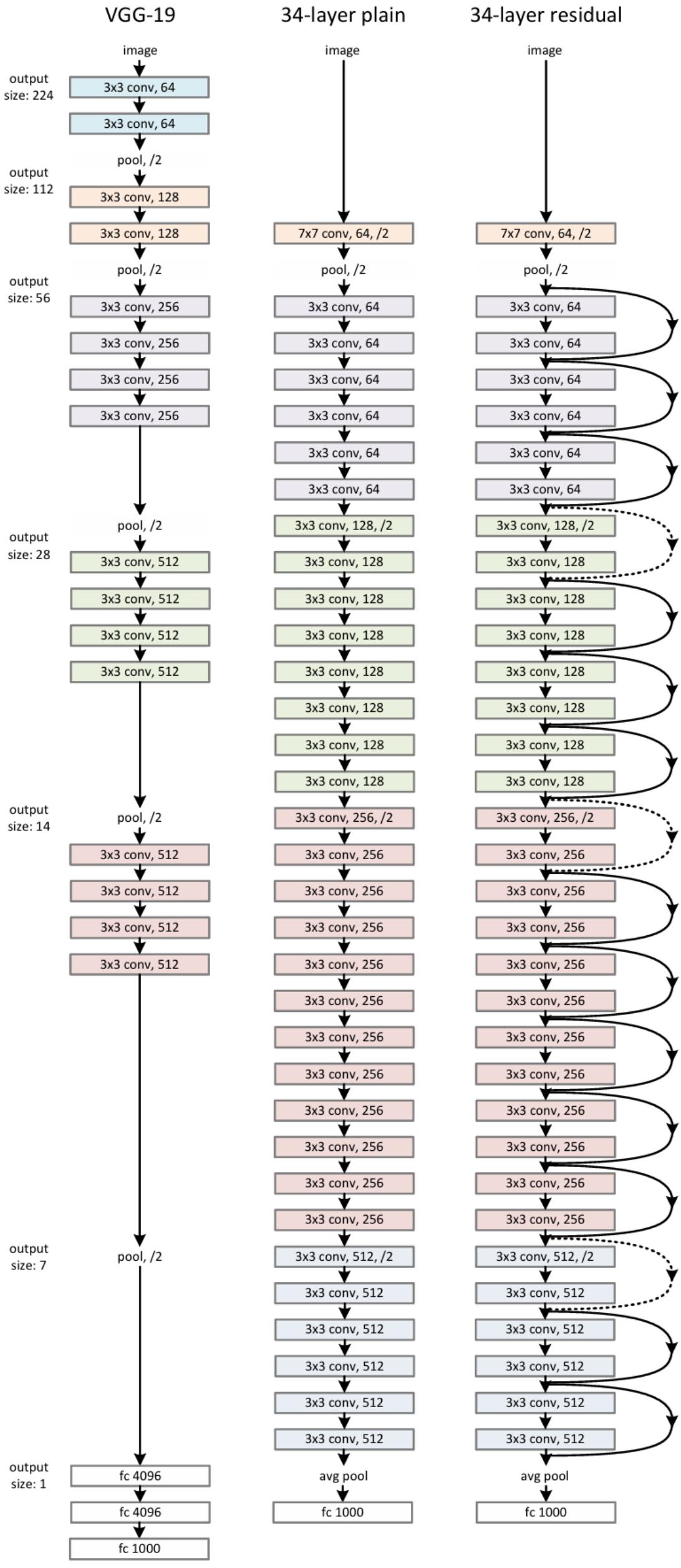
ImageNet Challenge result: 6.7% (1st)

Residual Blocks

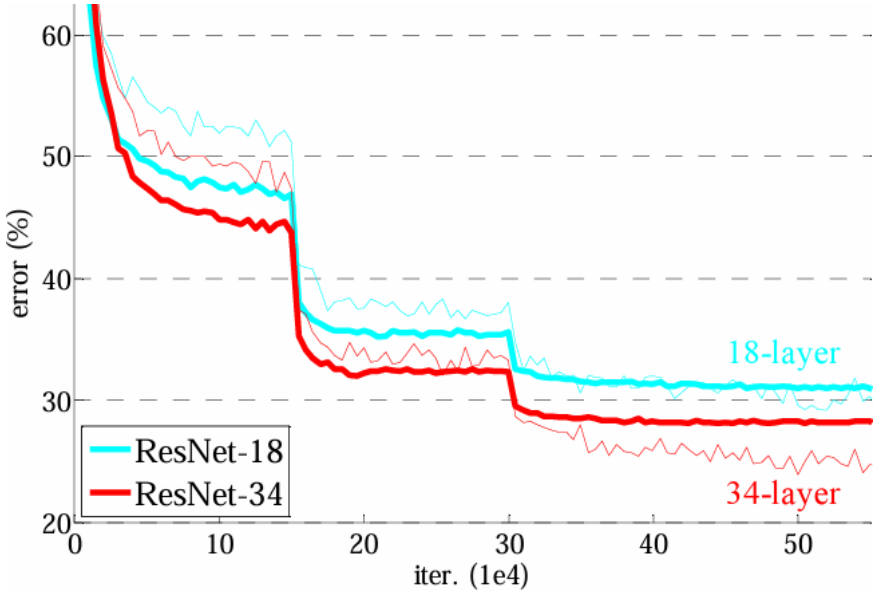


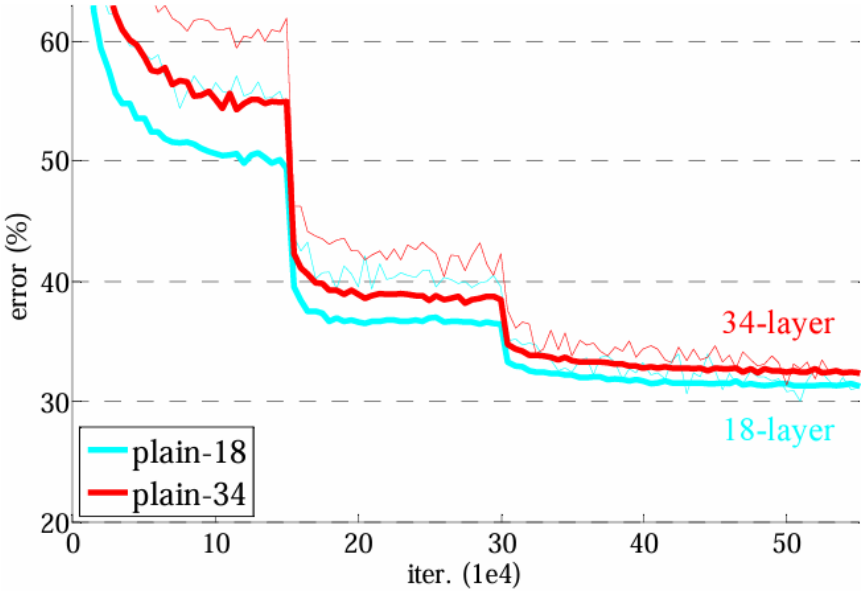
Regular block Residual block

Also called *skip-connection* or *shortcut-connection*

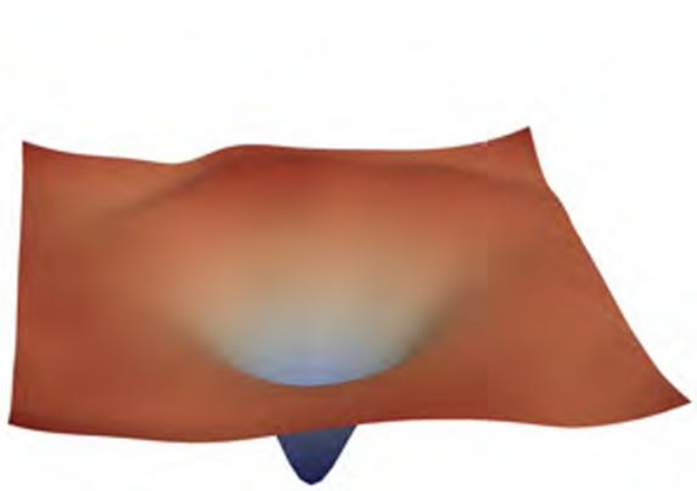

VGG-19
34-layer plain
34-layer residual
image
output size: 224
3x3 conv, 64
3x3 conv, 64
pool, /2
output size: 112
3x3 conv, 128
3x3 conv, 128
pool, /2
output size: 56
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
pool, /2
output size: 28
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
output size: 14
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
output size: 7
fc 4096
fc 4096
fc 1000
output size: 1
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
avg pool
fc 1000
ResNet (2015)

ResNet

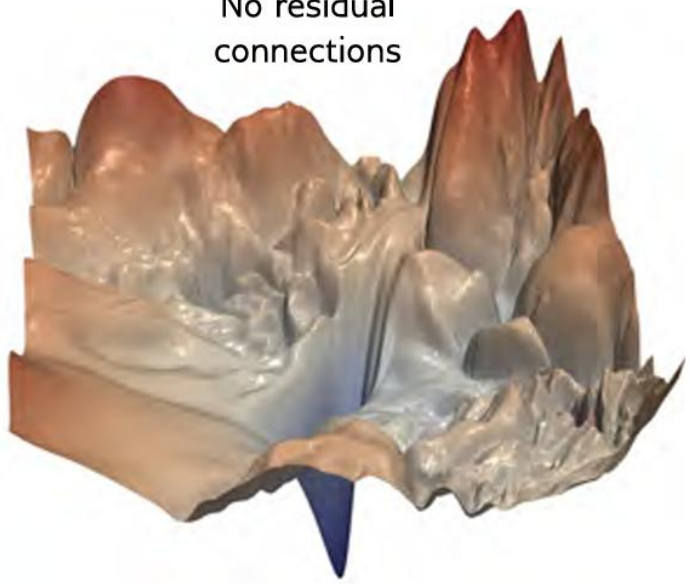




Residual connections



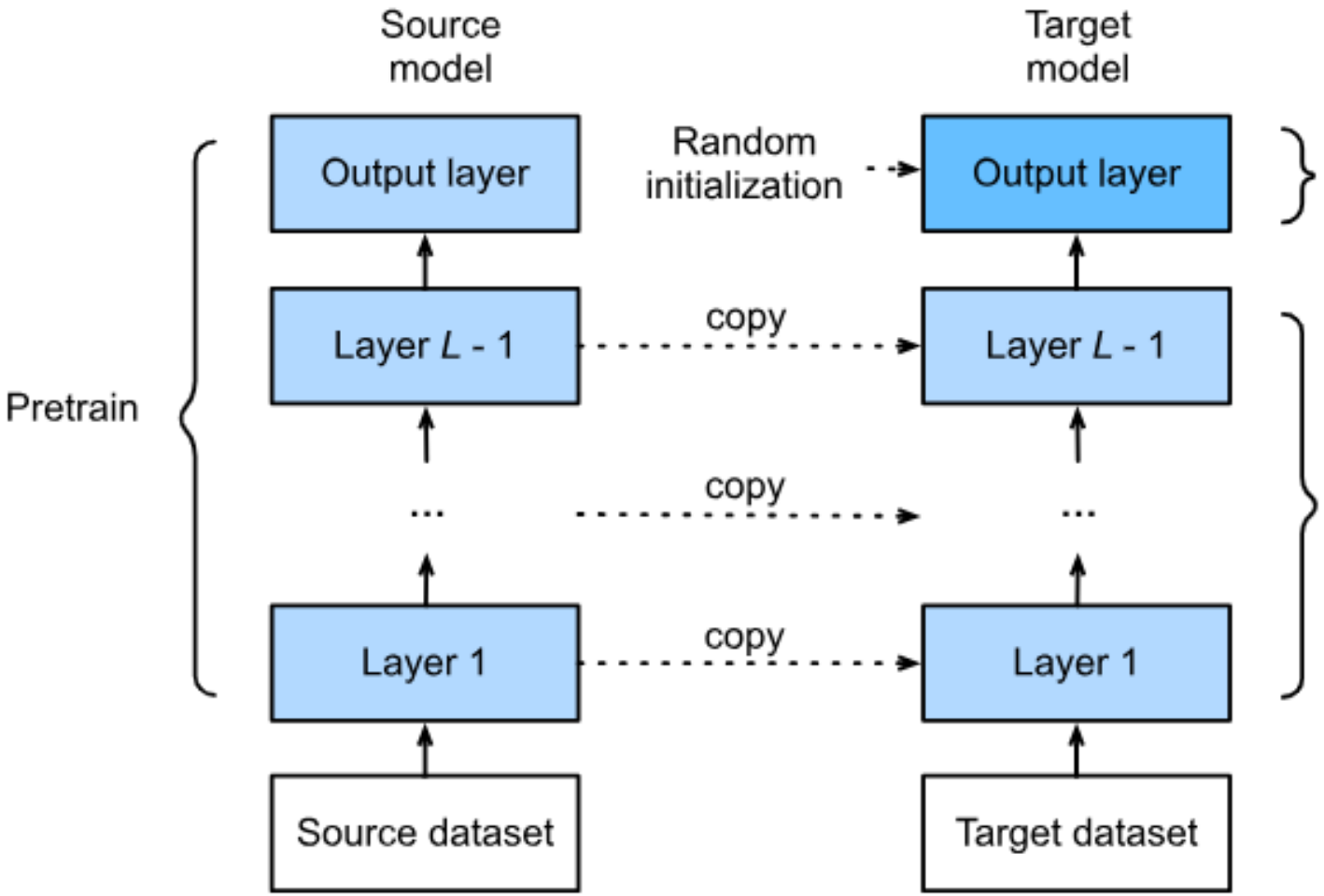
No residual connections



ResNet: Results

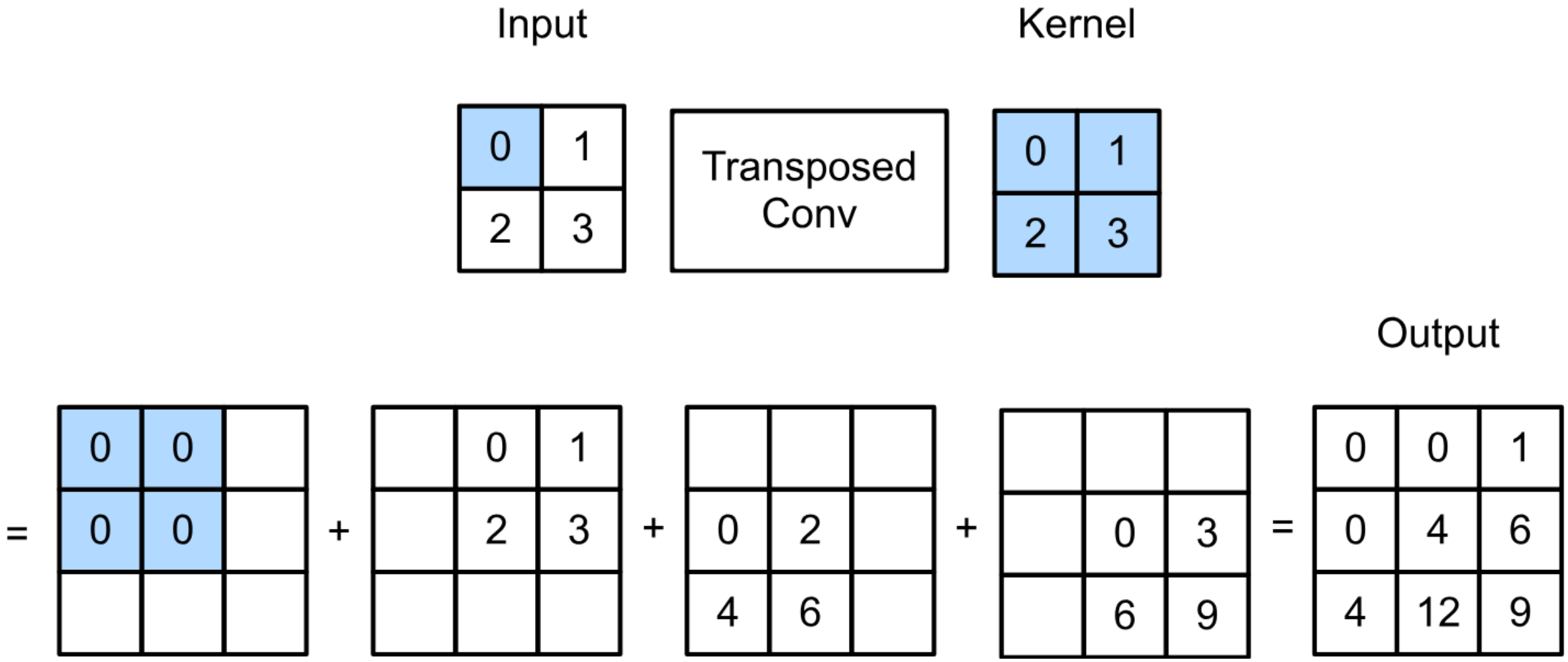
method	top-5 err. (**test**)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	**3.57**

Transfer Learning

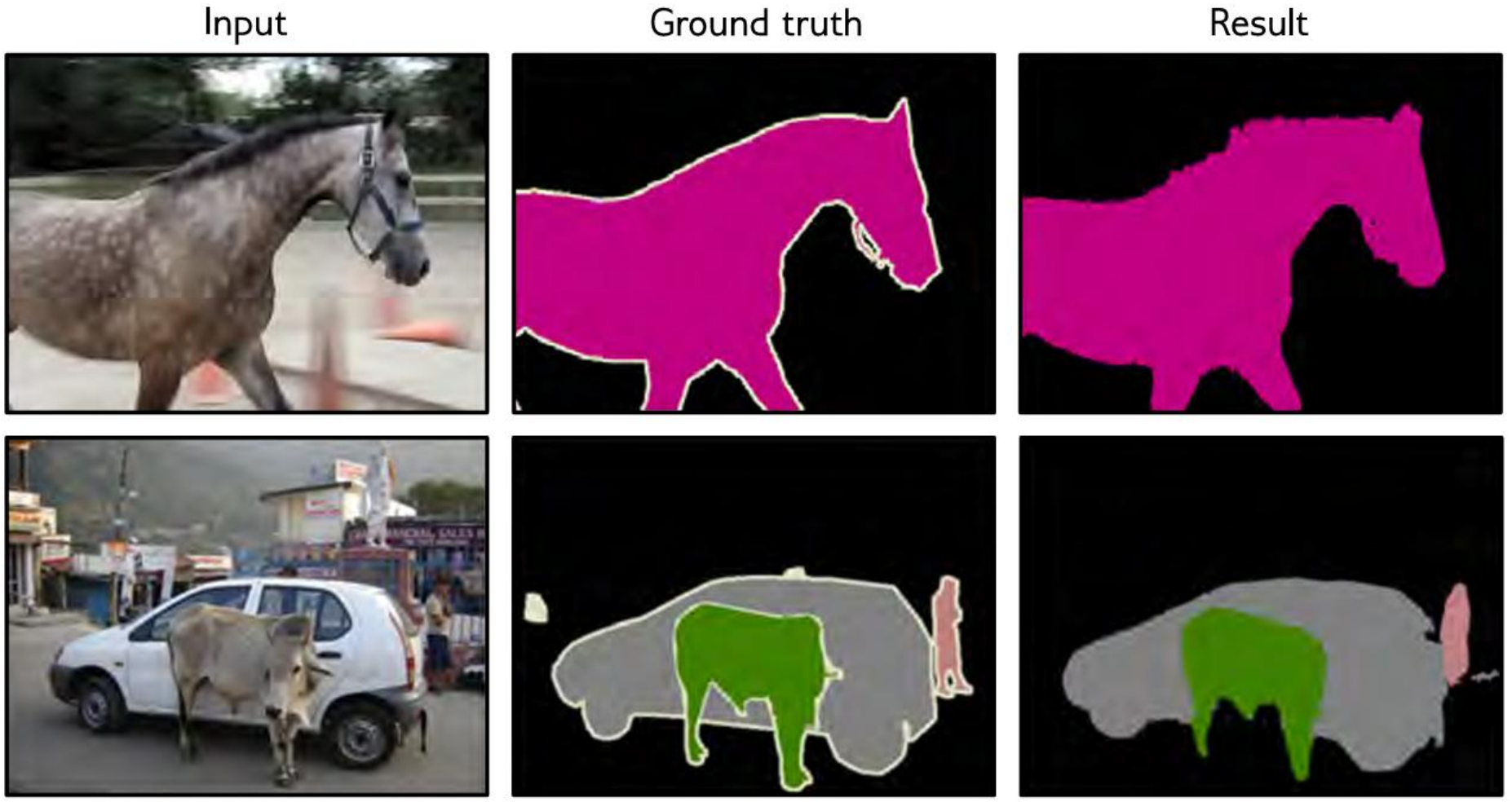


Trained in ***fine tuning***
Frozen in ***feature extraction***

Transposed Convolution

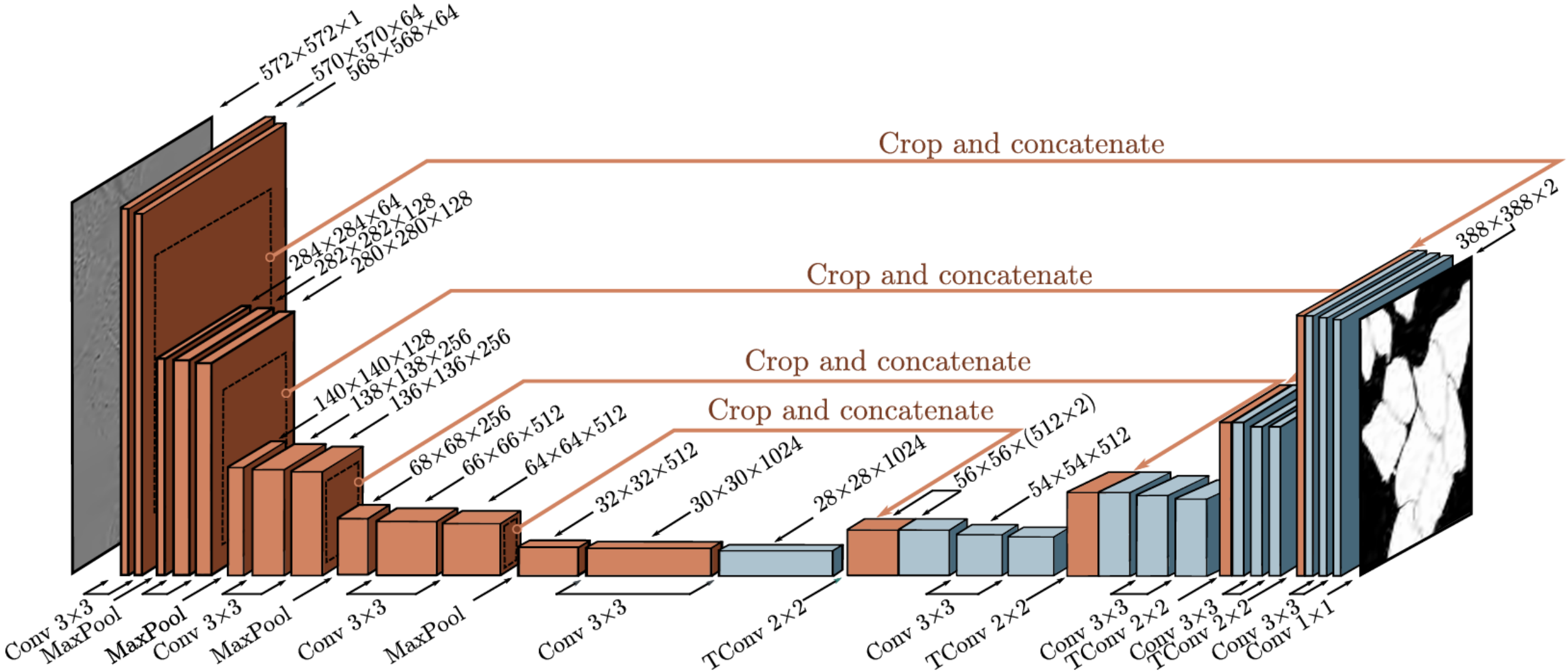


Segmentation



The goal of semantic segmentation is to assign a label to each pixel according to the object that it belongs to or no label if that pixel does not correspond to anything in the training database

U-Net


572×572×1
570×570×64
568×568×64
Crop and concatenate
284×284×64
282×282×128
280×280×128
Crop and concatenate
140×140×128
138×138×256
136×136×256
Crop and concatenate
68×68×256
66×66×512
64×64×512
Crop and concatenate
32×32×512
30×30×1024
28×28×1024
56×56×(512×2)
54×54×512
388×388×2
Conv 3×3
MaxPool
MaxPool
Conv 3×3
MaxPool
Conv 3×3
MaxPool
Conv 3×3
TConv 2×2
Conv 3×3
TConv 2×2
Conv 3×3
TConv 2×2
Conv 3×3
TConv 2×2
Conv 3×3
Conv 1×1